



POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



Các thành phần phần mềm

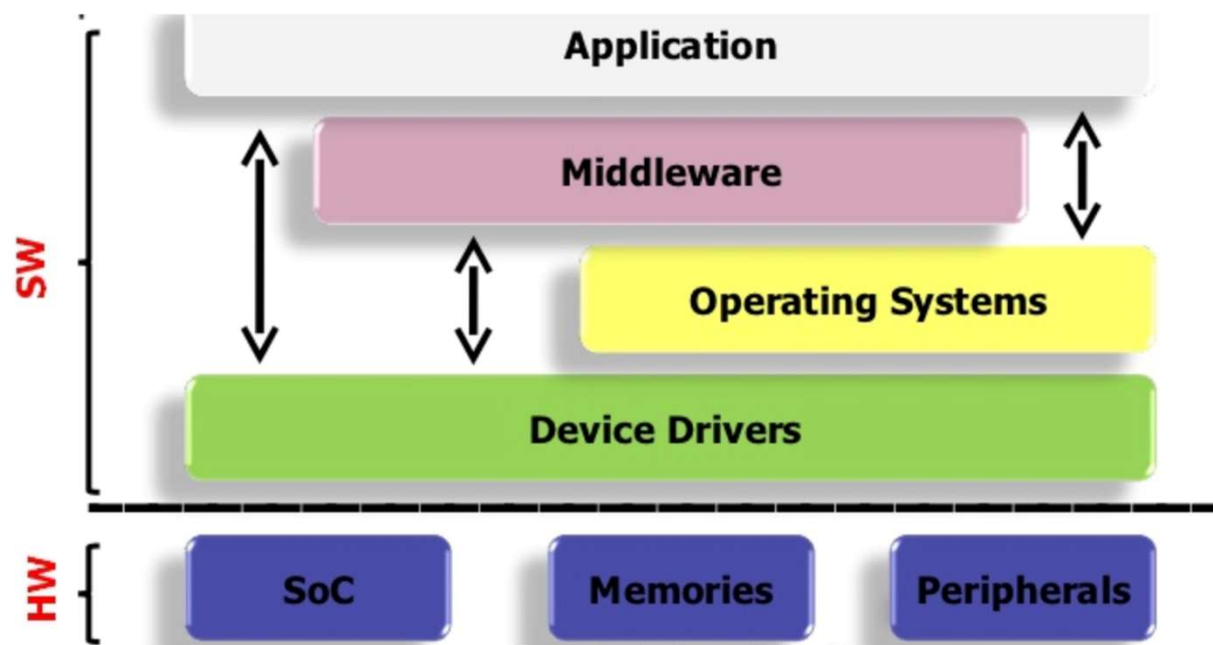
Faculty: Computer science – IT1

Instructor: Viet Hoang Pham

Email: vietph@ptit.edu.vn

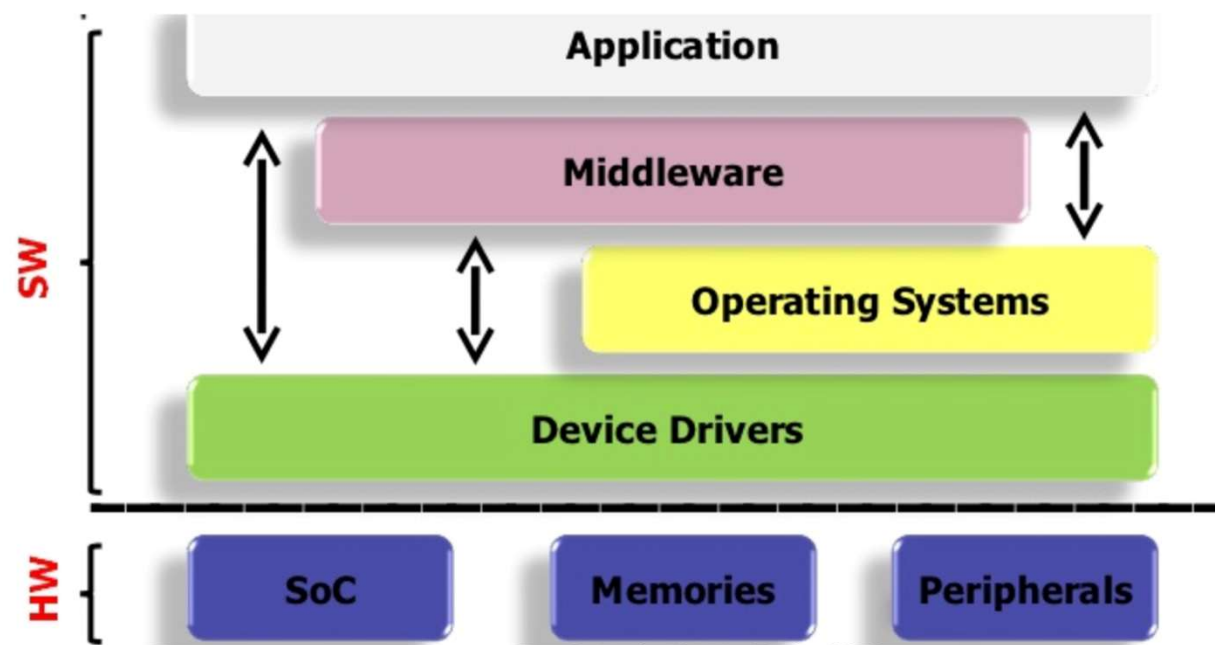
Phone: 0945296388

- Trình điều khiển thiết bị
- Hệ điều hành thời gian thực
- Middleware and Các phần mềm ứng dụng



Giới thiệu về phần mềm nhúng

- ❖ Phần mềm ứng dụng: định nghĩa chức năng cũng như mục đích hình thành một HTN cụ thể.
- ❖ Phần mềm hệ thống: quản lý hoạt động của phần cứng, cung cấp nguồn tài nguyên phần cứng và phần mềm trung gian khác cho phần mềm ứng dụng.



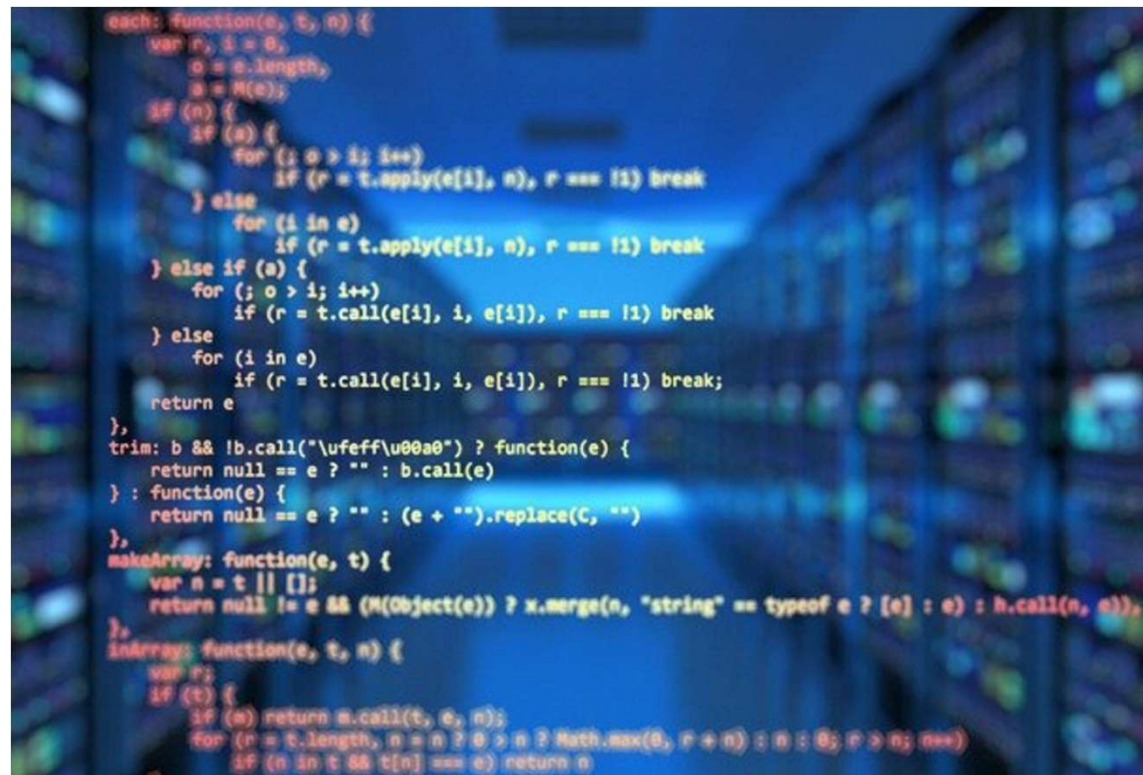
- ❖ Phần mềm hệ thống bao gồm các thành phần chính:
 - Hệ điều hành (Operating System - OS) hoặc Monitor
 - Nhân hệ điều hành (Kernel)
 - Trình điều khiển thiết bị (Device Driver - TĐKTB)
 - Phần mềm trung gian (Middleware)
 - Các thành phần hỗ trợ khởi động và gỡ lỗi

❖ **Gọi hệ thống (System Call - GHT)** là phương thức liên lạc giữa phần mềm hệ thống và phần mềm ứng dụng, cho phép ứng dụng khai thác các dịch vụ và chức năng do hệ điều hành (OS) cung cấp.

- Đây được coi là một giao diện lập trình (API) giúp tách biệt giữa môi trường người dùng và môi trường phần cứng máy tính.
- Nhân cung cấp các hàm chức năng cơ bản (primitive functions) để từ đó xây dựng nên các tiện ích và dịch vụ cao cấp hơn cho hệ thống.
- Gọi hệ thống giúp ngăn chặn việc các ứng dụng người dùng truy cập trái phép hoặc gây tranh chấp tài nguyên
- Ví dụ:
 - HĐH linux dung INT 0x80 để làm TRAP kích hoạt các dịch vụ nhân.
 - Chương trình C thực hiện lệnh read(): gọi hệ thống để yêu cầu nhân truy xuất dữ liệu từ thiết bị lưu trữ.
 - Lệnh fork() để tạo tiến trình mới được thực hiện thông qua cơ chế GHT

❖ Có hai cách tiếp cận để xây dựng phần mềm nhúng:

- Thiết kế phần mềm dựa trên các quy trình thông thường.
- Thiết kế dựa trên Hệ điều hành nhúng.



```
each: function(e, t, n) {  
    var r, i = 0,  
        o = e.length,  
        a = N(e);  
    if (n) {  
        if (a) {  
            for (; o > i; i++)  
                if (r = t.apply(e[i], n), r === !1) break;  
        } else  
            for (i in e)  
                if (r = t.apply(e[i], n), r === !1) break;  
    } else if (a) {  
        for (; o > i; i++)  
            if (r = t.call(e[i], i, e[i]), r === !1) break;  
    } else  
        for (i in e)  
            if (r = t.call(e[i], i, e[i]), r === !1) break;  
    return e;  
},  
trim: b && !b.call("\uffeff\u00a0") ? function(e) {  
    return null == e ? "" : b.call(e)  
} : function(e) {  
    return null == e ? "" : (e + "").replace(C, "")  
},  
makeArray: function(e, t) {  
    var n = t || [];  
    return null != e && (N(Object(e)) ? x.merge(n, "string" == typeof e ? [e] : e) : h.call(n, e)),  
},  
isArray: function(e, t, n) {  
    var r;  
    if (t) {  
        if (n) return m.call(t, e, n);  
        for (r = t.length, n = n ? 0 > n ? Math.max(0, r + n) : n : 0; r > n; n++)  
            if (n in t && t[n] === e) return n;  
    }  
}
```


Giới thiệu về phần mềm nhúng

❖ Có hai cách tiếp cận để xây dựng phần mềm nhúng:

Đặc điểm	Tiếp cận dựa trên thủ tục	Tiếp cận dựa trên Hệ điều hành
Hệ điều hành	Không cần.	Cần (GPOS hoặc RTOS).
Lập lịch	Tuần tự, cố định.	Theo mức ưu tiên, đa nhiệm.
Giá thành	Thấp, tiết kiệm bộ nhớ.	Cao hơn do yêu cầu tài nguyên.
Độ ổn định	Một lỗi có thể treo cả hệ thống.	Có khả năng cô lập và phục hồi lỗi.
Ứng dụng	Thiết bị đơn giản (đồ chơi, điều khiển cơ bản).	Thiết bị phức tạp (smartphone, router, y tế).

❖ Thiết kế phần mềm dựa trên các quy trình thông thường.

- Sử dụng hyperloop cho các ứng dụng không phụ thuộc vào thời gian phản hồi của hệ thống.
- Mỗi quy trình được thực hiện tuần tự:
 1. Cấu hình các tham số dùng chung và thực hiện khởi tạo các thành phần phần cứng như bộ nhớ và thanh ghi.
 2. Bắt đầu với tác vụ đầu tiên và thực thi nó.
Thực thi tác vụ thứ hai.
Thực thi tác vụ tiếp theo.
....
Thực thi tác vụ cuối cùng.
 3. Quay lại tác vụ đầu tiên và thực thi trong một luồng tương tự.

```
void main ()  
{  
    Configurations ();  
    Initialisations ();  
    while (1)  
    {  
        Task 1 ();  
        Task 2 ();  
        :  
        :  
        Task n ();  
    }  
}
```


❖ Thiết kế phần mềm dựa trên các quy trình thông thường.

■ Ưu điểm:

- Không cần hệ điều hành.
- Thiết kế đơn giản.
- Giá thành thấp.

■ Nhược điểm:

- Dễ bị ảnh hưởng bởi toàn bộ hệ thống nếu xảy ra lỗi.
- Thiếu thời gian xử lý

```
void main ()  
{  
    Configurations ();  
    Initialisations ();  
    while (1)  
    {  
        Task 1 ();  
        Task 2 ();  
        :  
        :  
        Task n ();  
    }  
}
```

Giới thiệu về phần mềm nhúng

❖ Thiết kế phần mềm dựa trên các quy trình thông thường.



- ❖ Thiết kế phần mềm dựa trên Hệ điều hành nhúng:
 - Một hệ điều hành chứa chính hệ điều hành đó.
 - Tạo và chạy các ứng dụng trên hệ điều hành.
 - Có thể là một hệ điều hành thông thường hoặc một hệ điều hành thời gian thực (RTOS).

❖ Thiết kế phần mềm dựa trên Hệ điều hành nhúng:

- Hệ điều hành thông thường:
 - thống nhất hệ thống nhúng với hệ thống máy tính thông thường của hệ điều hành;
 - hỗ trợ các giao diện lập trình có thể được sử dụng.
- Hệ điều hành thời gian thực:
 - các sản phẩm nhúng yêu cầu thời gian phản hồi;
 - chứa phần mềm chịu trách nhiệm ngăn chặn đa nhiệm, lập kế hoạch lịch trình thực thi, đa luồng;
 - một hệ điều hành thời gian thực cho phép lập lịch linh hoạt các tài nguyên hệ thống như CPU và bộ nhớ và xuất ra nhiều cách để giao tiếp giữa các tác vụ.

❖ Thiết kế phần mềm dựa trên Hệ điều hành nhúng:

■ Ưu điểm

- Môi trường phát triển thuận lợi
- Tái sử dụng mã nguồn
- Quản lý đa nhiệm và đa luồng
- Tính cô lập và bảo vệ
- Trừu tượng hóa phần cứng (HAL)

■ Khuyết điểm

- Đòi hỏi tài nguyên lớn
- Chi phí cao
- Độ phức tạp trong quản lý
- Tăng độ trễ hệ thống:

❖ Thiết kế phần mềm dựa trên Hệ điều hành nhúng:

Đặc điểm	Hệ điều hành thông thường (GPOS)	Hệ điều hành thời gian thực (RTOS)
Mục tiêu	Hợp nhất HTN với hệ tính toán chung, tận dụng thư viện có sẵn,.	Đáp ứng các yêu cầu về thời gian phản hồi tức thời và tính tiên định,.
Ưu tiên	Tập trung vào sự tiện dụng và đa dạng tính năng.	Tập trung vào việc đảm bảo hạn chót (deadline) và quản lý tài nguyên khẩn khe,.

Driver là gì



- ❖ Trình điều khiển thiết bị (TĐKTB - Device Driver) được định nghĩa là một loại phần mềm đặc biệt dùng để khởi động phần cứng và cung cấp giao diện để các phần mềm lớp cao hơn có thể quản trị việc truy nhập vào các thành phần phần cứng được ghép nối với hệ thống.
 - TĐKTB đóng vai trò là cầu nối giữa phần cứng và hệ điều hành.
 - Đây là lớp phần mềm tương tác trực tiếp nhất với các thanh ghi điện tử và cấu trúc vật lý của phần cứng, thường được tổ chức dưới dạng các thư viện phần mềm.
 - Hệ điều hành thường không quản lý trực tiếp thiết bị mà nhìn chúng thông qua một lớp phần cứng trừu tượng (HAL). Các thiết bị được ảo hóa thông qua trình điều khiển thiết bị ảo (virtual device drivers)

❖ Trình điều khiển thiết bị (TĐKTB - Device Driver)

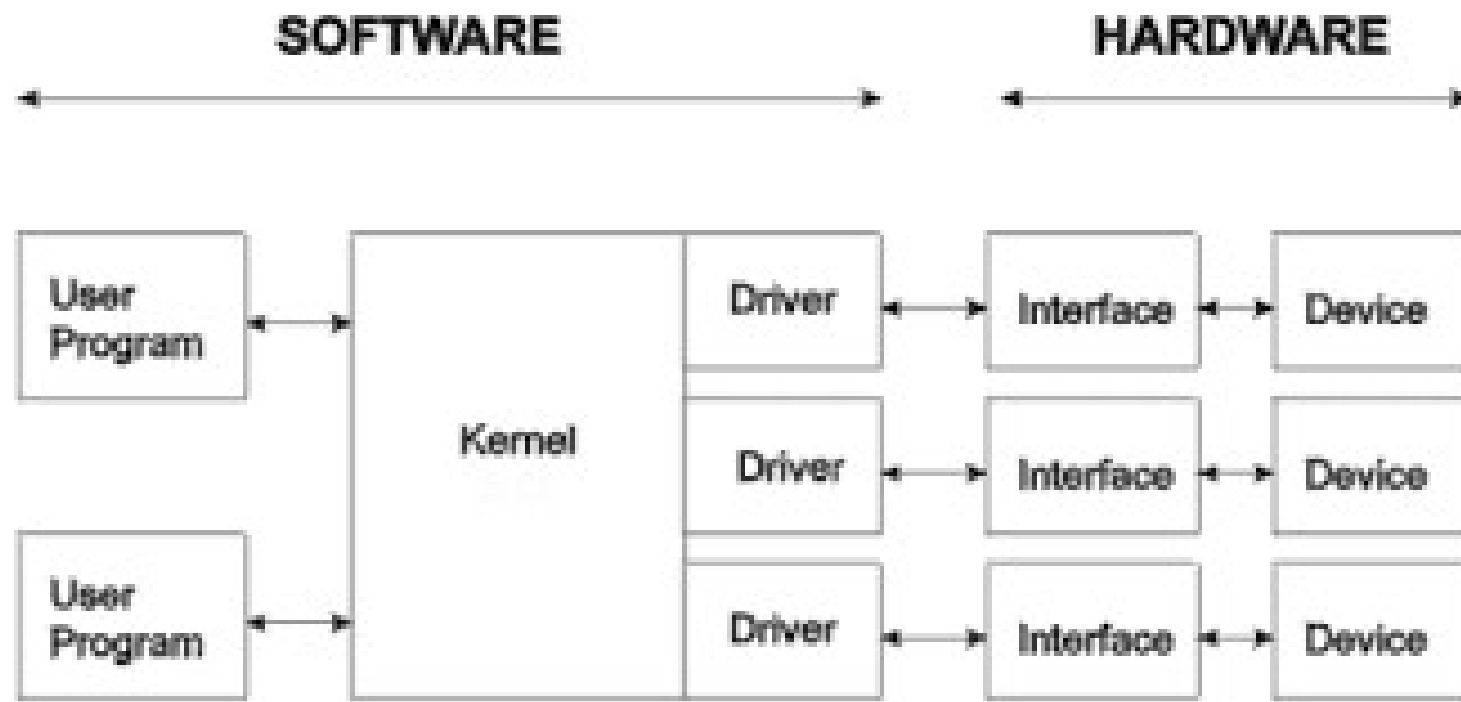
- Một trình điều khiển thiết bị điển hình thường đảm nhiệm các chức năng sau
 - Khởi động (Startup) và Tắt (Shutdown),
 - Cấm (Disable) và Cho phép (Enable),
 - Chiếm dụng (Acquire) và Giải phóng (Release),
 - Đọc/Ghi dữ liệu (Read/Write),
 - Cài đặt và tháo dỡ (Install/Uninstall)
- TĐKTB thường được chia thành hai lớp chính:
 - Lớp xác định theo kiến trúc (Architecture-specific): Quản trị các phần cứng tích hợp sâu vào CPU như bộ nhớ trên chip, đơn vị quản lý bộ nhớ (MMU), các bộ ADC/DAC nội bộ.
 - Lớp tổng quát (Generic): Điều khiển các thiết bị nằm trên bo mạch chính nhưng không hợp nhất vào chip CPU

❖ Định nghĩa trình điều khiển thiết bị (device driver):

- Device driver: **kết nối giữa OS và các thiết bị I/O.**
- Chức năng: dịch, convert các yêu cầu từ OS thành các câu lệnh mà bộ điều khiển ngoại vi có thể hiểu.
- Là một phần của nhân OS (drivers là các module phần mềm được xây dựng riêng biệt và cài vào OS khi cần thiết).
 - Các ví dụ: driver máy in, Bluetooth driver ...

❖ Định nghĩa trình điều khiển thiết bị (device driver):

- Một hệ thống hoàn chỉnh có rất nhiều driver được cài đặt trên hệ điều hành.
- Device drivers hoạt động ở chế độ đặc quyền (mode privileged): cần được thiết kế, bảo mật tốt, tránh mã độc.



- ❖ Ưu điểm sử dụng trình điều khiển thiết bị (device driver):
 - Đơn giản hóa hoạt động của OS
 - Không có device driver:
 - OS chịu trách nhiệm giao tiếp trực tiếp với phần cứng
→ gây quá tải cho OS
 - Nếu một thiết bị mới được cài đặt, dẫn đến phải thay đổi OS.

❖ Các vấn đề thiết kế chính:

- Giao tiếp giữa Hệ điều hành và Driver (OS/Driver Communication)
- Giao tiếp giữa Driver và Phần cứng (Driver/Hardware Communication)
- Hoạt động nội tại của Driver (Driver Internal Operations)

❖ Các vấn đề thiết kế chính:

- Giao tiếp giữa hệ điều hành (OS) và trình điều khiển thiết bị (Driver)
 - Trao đổi thông tin (command, data)
 - Tiếp nhận yêu cầu (Commands)
 - Chuyển đổi lệnh
 - Trao đổi dữ liệu (Data)
 - Các hàm hỗ trợ mà kernel cung cấp
 - Lớp phần cứng trừu tượng (HAL): cho phép OS hoạt động độc lập tương đối với các đặc thù vật lý của từng loại phần cứng khác nhau.
 - Quản lý tài nguyên hệ thống: Quản lý bộ nhớ, Xử lý ngắt, Lập lịch
 - Truyền thông liên tiến trình (IPC): Cung cấp các cơ chế đồng bộ và liên lạc như Semaphore, Mailbox, hoặc hàng đợi thông điệp
 - Dịch vụ thời gian: đồng bộ hóa các thao tác điều khiển phần cứng.

❖ Các vấn đề thiết kế chính:

- Giao tiếp giữa Driver và Phần cứng (Driver / Hardware Communication)
 - Trao đổi thông tin (command, data),
 - Phần mềm giao tiếp với Phần cứng,
 - Phần cứng giao tiếp với Phần mềm

❖ Các vấn đề thiết kế chính:

- Driver Internal Operations
 - Dịch command nhận được từ OS,
 - Sắp xếp thứ tự các yêu cầu,
 - Quản lý truyền dữ liệu giữa các interfaces (OS và hardware),
 - Chấp nhận và xử lý các ngắt,
 - Duy trì sự thống nhất của cấu trúc dữ liệu giữa driver và kernel.

❖ Các loại trình điều khiển thiết bị (Driver):

- Trình điều khiển thiết bị kiểu khối (Block Driver)
- Trình điều khiển thiết bị kiểu ký tự (Character Driver)
- Trình điều khiển thiết bị đầu cuối (Terminal Driver)
- Trình điều khiển thiết bị kiểu luồng (Stream Driver)

❖ Các loại driver:

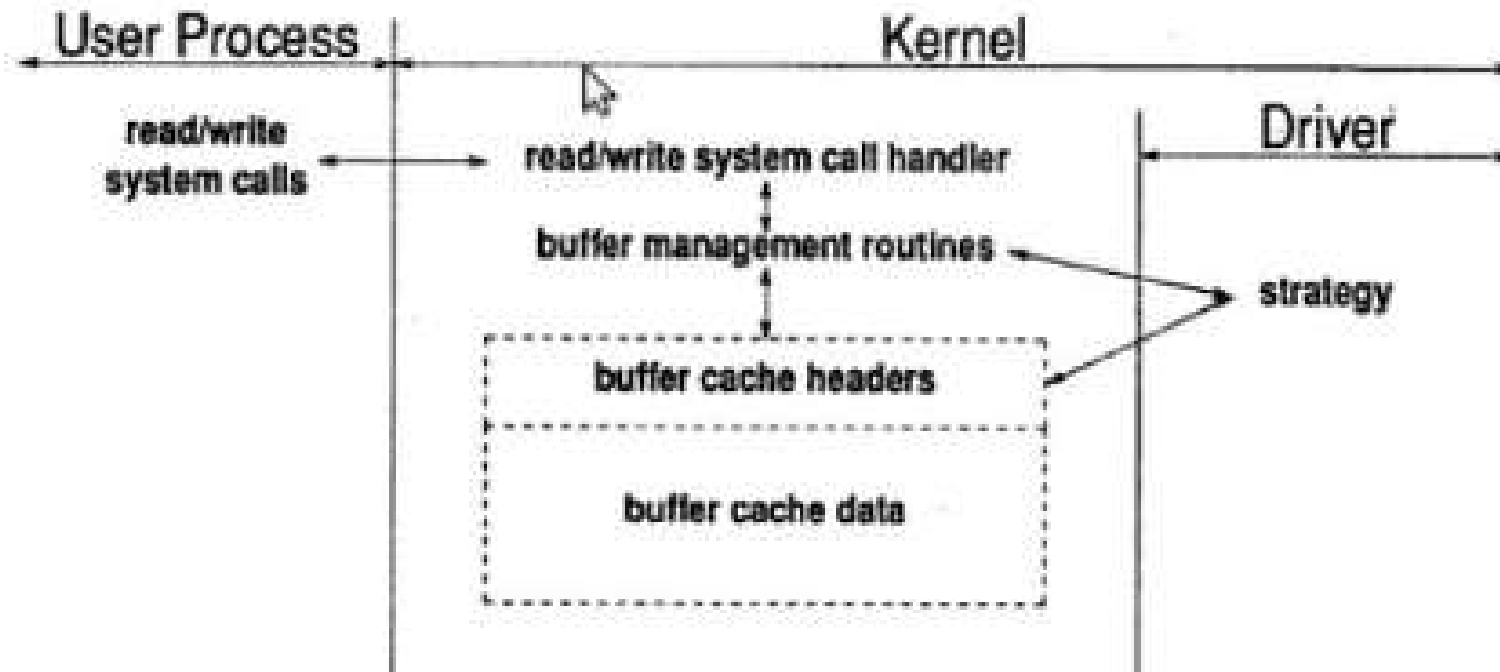
Loại Driver	Đơn vị dữ liệu	Yêu cầu Buffer	Phương thức truyền phổ biến
Block	Khối (Block)	Lớn	DMA
Character	Ký tự (Byte)	Nhỏ/Không	Cổng I/O, Ngắt
Stream	Luồng (Stream)	Trung bình	Nối tiếp (Serial)
Terminal	Lệnh/Ký tự	Thấp	CLI qua Serial Port

❖ Các loại driver:

- Trình điều khiển thiết bị kiểu khối (Block Driver)
 - Giao tiếp với OS thông qua tập các bộ đệm với kích thước cố định (block of data).
 - OS quản lý bộ nhớ cache của các bộ đệm và đáp ứng yêu cầu người dùng về dữ liệu bằng cách truy cập vào cache.
 - Driver chỉ được gọi khi các dữ liệu yêu cầu không có trong cache hoặc khi bộ đệm trong cache thay đổi.
 - Driver làm việc với các yêu cầu từ OS để lấp đầy hoặc xóa các bộ đệm với kích thước cố định.

❖ Các loại driver:

- Trình điều khiển thiết bị kiểu khối (Block Driver)
 - Ví dụ:

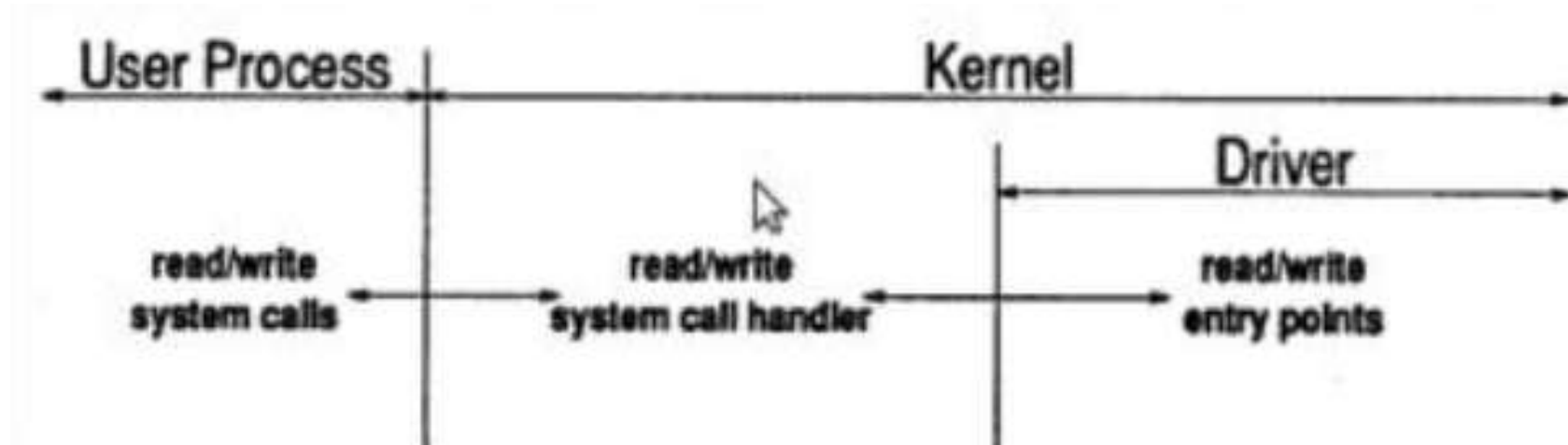


❖ Các loại driver:

- Trình điều khiển thiết bị kiểu ký tự (Character Driver)
 - Các yêu cầu với kích thước gói tin độ dài bất kì của I/O.
 - Được dùng cho các thiết bị giao tiếp theo byte tại 1 thời điểm hoặc với khối data lớn hoặc nhỏ hơn khối đệm dữ liệu của block drivers.
 - VD: máy in
 - Các yêu cầu từ I/O được chuyển trực tiếp đến driver và driver chuyển dữ liệu đến bộ nhớ VXL.

❖ Các loại driver:

- Trình điều khiển thiết bị kiểu ký tự (Character Driver)
 - Ví dụ:



❖ Các loại driver:

Block Driver	Character Driver
Truyền dữ liệu đến kernel buffer cache	Truyền dữ liệu trực tiếp đến tiến trình người dùng
Không phù hợp cho các thiết bị giao tiếp với gói tin kích thước thay đổi	Phù hợp với các thiết bị giao tiếp theo gói tin có kích thước thay đổi
Có thể được dùng để hỗ trợ thiết bị chạy trên nền UNIX	Không hỗ trợ thiết bị chạy nền UNIX
Phù hợp hỗ trợ các ổ đĩa, ổ cứng	Phù hợp hỗ trợ cổng nối tiếp, cổng song song, card mạng, bàn phím...

❖ Các loại driver:

- Trình điều khiển thiết bị đầu cuối (Terminal Driver)
 - thường sử dụng đường kết nối nối tiếp (Serial link) để thiết lập kênh liên lạc giữa hệ thống nhúng đích và hệ thống chủ phát triển.
 - thực hiện các thao tác quản trị hệ thống trực tiếp bao gồm:
 - Tải mã nguồn
 - Truy cập bộ nhớ
 - Quản lý thanh ghi
 - Điều khiển hệ thống
 - có khả năng hỗ trợ gỡ lỗi (debugging)

❖ Các loại driver:

- Trình điều khiển thiết bị kiểu luồng (Stream Driver)
 - quản lý các thiết bị ngoại vi hỗ trợ truyền dẫn dữ liệu dưới dạng một chuỗi byte liên tục (byte stream).
 - tập trung vào tính tuần tự và thông suốt của dữ liệu.
 - thứ tự của các byte dữ liệu là tối quan trọng và phải được duy trì chính xác từ nguồn gửi đến đích nhận.
 - hệ thống thường sử dụng các bộ đệm FIFO (First-In-First-Out)
 - Đối với luồng dữ liệu tốc độ cao, Driver phải có mã xử lý ngắt (ISR) tối ưu để phản ứng kịp thời khi dữ liệu sẵn sàng

❖ Các loại driver:

- Trình điều khiển thiết bị kiểu luồng (Stream Driver)
 - Việc viết mã nguồn để quản lý hàng đợi dữ liệu trong các Stream Driver đòi hỏi sự chính xác cao trong việc điều phối luồng dữ liệu liên tục và đảm bảo thứ tự của các byte
 - Phải thiết kế bộ đệm đủ lớn để tránh tình trạng tràn dữ liệu
 - Việc quản lý các con trỏ vào/ra (head/tail) của bộ đệm phải thật chính xác để tránh bị đảo lộn thứ tự của dữ liệu.
 - đảm bảo các thao tác đọc/ghi hàng đợi không bị xung đột khi có ngắt xảy ra.
 - Các bước lập trình quản lý hàng đợi
 - Khởi tạo và Làm sạch (Initialization)
 - Xử lý ghi dữ liệu (Input)
 - Xử lý đọc dữ liệu (Output)
 - Kiểm tra trạng thái

❖ Đặc điểm của driver:

- Driver là tập các entry points được gọi bởi OS.
- Một driver bao gồm:
 - Cấu trúc data riêng biệt cho từng thiết bị.
 - Giao thức riêng biệt cho từng driver
- Hầu hết các drivers được viết trên 1 file nguồn.
- Phần đầu của driver được gọi là khai báo phần đầu (prologue).

❖ Đặc điểm của driver:

- Khai báo (prologue) gồm:
 - #include
 - #define
 - Khai báo các biến và đặc điểm các biến
- Phần còn lại của driver:
 - Entry points (các hàm C được tham chiếu bởi OS)
 - Routines (các hàm C tương tác với driver)

- ❖ Hoạt động của Trình điều khiển thiết bị (TĐKTB/ Device Driver):
 - Phần mềm ứng dụng không truy cập trực tiếp phần cứng mà thực hiện các lời gọi hệ thống (System Calls) đến HĐH.
 - TĐKTB nhận các thông số từ HĐH, sau đó tương tác trực tiếp với các thanh ghi điện tử và cấu trúc vật lý của thiết bị để thực hiện lệnh.
 - Các giai đoạn chính khi một TĐKTB được kích hoạt trong hệ thống
 - Tự cấu hình (Autoconfiguration)
 - Thực hiện thao tác Vào/Ra (I/O Operations)
 - Xử lý ngắt (Interrupt Handling)
 - Tái khởi động (Reinitialization):

❖ Hoạt động của Trình điều khiển thiết bị (TĐKTB/ Device Driver):

- Trình điều khiển giám sát ít nhất ba trạng thái của phần cứng:
 - Không hoạt động (Idle)
 - Đang hoạt động (Busy)
 - Kết thúc hoạt động (Finish)
- Để đảm bảo tính ổn định trong môi trường đa nhiệm, nó cung cấp các chức năng quản trị tài nguyên:
 - Chiếm dụng (Acquire): Khóa thiết bị để dành quyền độc quyền sử dụng cho một tiến trình nhất định
 - Giải phóng (Release): Mở khóa thiết bị sau khi hoàn thành tác vụ
 - Cấm/Cho phép (Disable/Enable): Vô hiệu hóa hoặc kích hoạt hoạt động của phần cứng theo yêu cầu của hệ thống
- Trong các kiến trúc CPU hiện đại, trình điều khiển thiết bị thường chạy trong chế độ nhân (kernel mode/supervisor mode).

- ❖ Thiết kế và phát triển Trình điều khiển thiết bị (TĐKTB) :
 - Thu thập thông tin nền tảng
 - Thiết kế cấu trúc trình điều khiển
 - Quản lý cấu trúc dữ liệu và địa chỉ
 - Sắp đặt cấu trúc dữ liệu hệ thống
 - Tạo môi trường phát triển (DDK)

❖ Thiết kế và phát triển Trình điều khiển thiết bị (TĐKTB) :

■ Thu thập thông tin nền tảng

- Thông tin về máy chủ (Host): Xác định loại CPU, hệ điều hành (HĐH) đang sử dụng và kiến trúc BUS của hệ thống.
- Quy định viết mã: Nắm rõ các quy tắc đặt tên thiết bị, cách mô tả thiết bị và các chuẩn tài liệu liên quan.
- Thuộc tính thiết bị: Xác định thiết bị thuộc kiểu khối (block) hay kí tự (character); có hỗ trợ hệ thống tệp hay chuỗi dữ liệu (byte stream) không; và cách thức phản ứng với ngắt (cắm ngắt đệ qui, thao tác hàng đợi...).
- Mô tả chi tiết: Kiểu thiết bị, số lượng thiết bị cùng loại có thể cài đặt và mục đích sử dụng cụ thể.

- ❖ Thiết kế và phát triển Trình điều khiển thiết bị (TĐKTB) :
 - Thiết kế cấu trúc trình điều khiển
 - Xác định kiểu thiết bị: Phân loại rõ là thiết bị Kí tự, Khối, Mạng hoặc kết hợp cả hai.
 - Giao diện tiếp cận: Xác định địa chỉ cổng sử dụng và phương pháp để hệ thống nhận biết sự hiện diện của thiết bị (thông qua thăm dò - *probe* hoặc quét - *scan*).

❖ Thiết kế và phát triển Trình điều khiển thiết bị (TĐKTB) :

- Quản lý cấu trúc dữ liệu và địa chỉ
 - Kiểu dữ liệu thao tác: Tùy thuộc vào cách ghép vào BUS hệ thống để xác định dữ liệu là song song, nối tiếp, nhị phân hay hexa.
 - Địa chỉ ảo và địa chỉ I/O: Xác định địa chỉ ảo trong không gian HĐH đa nhiệm và địa chỉ cổng vật lý ghép nối tới thiết bị,.
 - Phương pháp đăng ký ngắt: Quyết định cách thiết bị đăng ký với module quản lý ngắt (interrupt handler), thông qua lời gọi hệ thống hay nhảy trực tiếp vào bảng vector ngắt.

- ❖ Thiết kế và phát triển Trình điều khiển thiết bị (TĐKTB) :
 - Sắp đặt cấu trúc dữ liệu hệ thống: xác định vị trí cài đặt các cấu trúc dữ liệu về thiết bị khi hệ thống khởi động, cũng như nơi đặt con trỏ trỏ tới Chương trình dịch vụ ngắt (ISR).
 - Tạo môi trường phát triển (DDK)
 - Các hãng sản xuất CPU thường cung cấp các bộ DDK (Device Driver Development Kit) đi kèm.
 - Cài đặt tổ chức thư mục để biên dịch, tạo mã thực thi, sau đó hợp nhất vào HĐH và nạp xuống hệ thống nhúng đang phát triển.

❖ Một số ví dụ:

- Các hệ điều hành thương mại đã tích hợp sẵn TĐKTB cho các thiết bị chuẩn như:
 - Thiết bị lưu trữ: Đĩa cứng, đĩa mềm, ổ đĩa CD, DVD.
 - Thiết bị ngoại vi giao tiếp: Bàn phím, màn hình.
 - Thiết bị mạng: Vỉ mạng (Network card).

❖ Một số ví dụ:

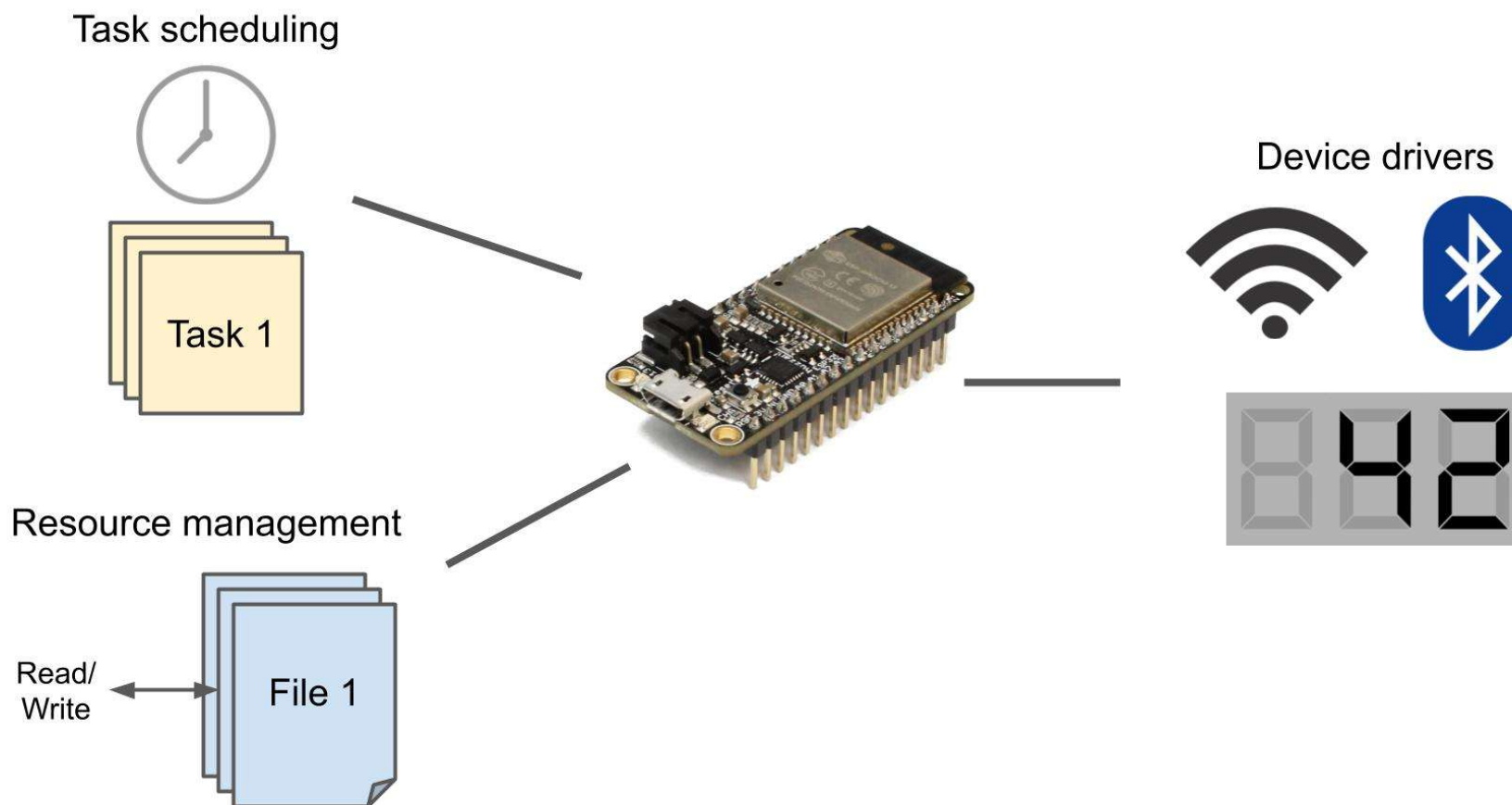
■ TĐKTB theo chức năng chuyên biệt

- TĐKTB điều khiển ngắt: gửi lệnh điều khiển tới vi mạch quản lý ngắt, cài đặt bảng vector ngắt và kích hoạt quyền nhận ngắt sau khi bật máy.
- TĐKTB điều khiển BUS: Quản lý giao thức (protocol) của BUS hệ thống, bao gồm việc khởi động/tắt BUS, khóa/giải khóa BUS để độc quyền sử dụng, và thực hiện các quy trình "bắt tay" khi trao đổi dữ liệu.
- TĐKTB cho bo mạch cắm thêm: Điều khiển các bo mạch được thiết kế riêng cho chức năng mới khi cắm vào khe I/O của hệ thống, quản lý việc khởi động và trao đổi dữ liệu giữa bộ xử lý chủ (master) và vi mạch trên bo cắm thêm (slave).

❖ Một số ví dụ:

- TĐKTB phân loại theo cách thức xử lý dữ liệu
 - Thiết bị cung cấp khối dữ liệu (Block device): TĐKTB cho các thiết bị như máy MP3, máy quay phim (Camcorder), TV, hoặc dữ liệu từ các cảm biến radar. Những loại này thường sử dụng phương thức DMA để truyền dữ liệu.
 - Thiết bị xử lý ký tự (Character device): TĐKTB cho các cảm biến gửi dữ liệu qua bộ chuyển đổi ADC/DAC, cổng truyền thông nối tiếp, hoặc cổng lập trình và gỡ lỗi SPI.
 - Thiết bị mạng (Network device): Chịu trách nhiệm nhận dữ liệu từng bit từ đường truyền, chuyển thành byte trong bộ đệm và đưa vào RAM thông qua cơ chế ngắt và DMA.

Real-Time Operating System (RTOS)



❖ Một số khái niệm:

- Nhân HĐH
- Tiến trình
- chế độ hoạt động
- chức năng
- Ngắt
- tác vụ
- cờ hiệu (semaphorre),
- Ngắt
- ngoại lệ ...

❖ Một số khái niệm:

■ Nhân HĐH (Kernel)

- Định nghĩa: Là phần mã cốt yếu nhất của các chương trình hệ thống, đóng vai trò "bộ não" điều khiển trực tiếp các tài nguyên vật lý của máy tính.
- Chức năng: Thực hiện quản lý tiến trình (tạo, giám sát), quản lý tài nguyên (CPU, RAM, I/O, File system), giải quyết tranh chấp tài nguyên và cung cấp các hàm chức năng nền tảng (primitive functions) cho ứng dụng qua giao diện gọi hệ thống (GHT).

❖ Một số khái niệm:

■ Tiến trình (Process)

- Định nghĩa: Là thời điểm thực hiện của một chương trình (instance of execution) trong một khoảng thời gian nhất định (slice time).
- Thành phần: Một tiến trình bao gồm mã lệnh (code), dữ liệu (data), và ngăn xếp (stack) riêng.
- Các trạng thái cơ bản:
 - Chạy (Run): Các lệnh của chương trình đang được thực hiện.
 - Dừng/Đợi (Suspended/Wait): Tiến trình tạm nghỉ do hết thời gian sử dụng CPU hoặc chờ tài nguyên/sự kiện.
 - Sẵn sàng (Ready): Đã có đủ tài nguyên và đang nằm trong hàng đợi để được hệ lập lịch kích hoạt chạy lại.

❖ Một số khái niệm:

■ Chế độ hoạt động (Operating Modes)

- Hệ điều hành đa nhiệm phân chia không gian hoạt động thành hai chế độ chính để đảm bảo an toàn hệ thống:
- Chế độ Nhân (Kernel/Supervisor Mode): Cho phép phần mềm truy nhập không hạn chế vào tài nguyên phần cứng. Mã của HĐH và các trình điều khiển thiết bị (TĐKTB) thường chạy ở chế độ này.
- Chế độ Người dùng (User Mode): Mã ứng dụng chạy trong không gian này và bị ngăn cản truy cập trực tiếp vào phần cứng để tránh làm hỏng hệ thống. Khi cần tài nguyên, ứng dụng phải qua GHT để yêu cầu nhân thực hiện thay.

❖ Một số khái niệm:

■ Chức năng (Function)

- Định nghĩa: Là một tập hợp các lệnh được tổ chức để thực hiện một hành động cụ thể.
- Đặc điểm:
 - Trong RTOS, chức năng được kích hoạt bởi các thủ tục, tác vụ hoặc chương trình phục vụ ngắt (ISR).
 - Mỗi chức năng có một bối cảnh thực thi (execution context) riêng.

❖ Một số khái niệm:

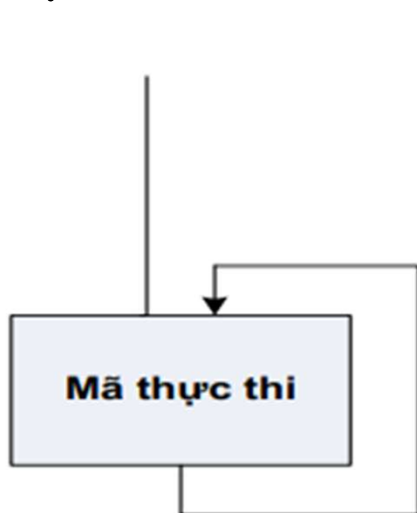
■ Tác vụ (Task)

- Định nghĩa: Là một ứng dụng hoặc một đơn vị công việc độc lập chạy trong thời gian thực, chịu sự giám sát của bộ lập lịch.
- Cấu trúc: Mỗi tác vụ bao gồm mã nguồn chức năng, một ngăn xếp (stack) riêng để lưu ngữ cảnh và có thể có hộp thư (mail box) để giao tiếp.
- Cơ chế: RTOS chia nhỏ các vấn đề phức tạp thành nhiều tác vụ chạy "song song" (thực tế là chia sẻ CPU) để phản ứng kịp thời với các sự kiện.

❖ Một số khái niệm:

■ Tác vụ (Task)

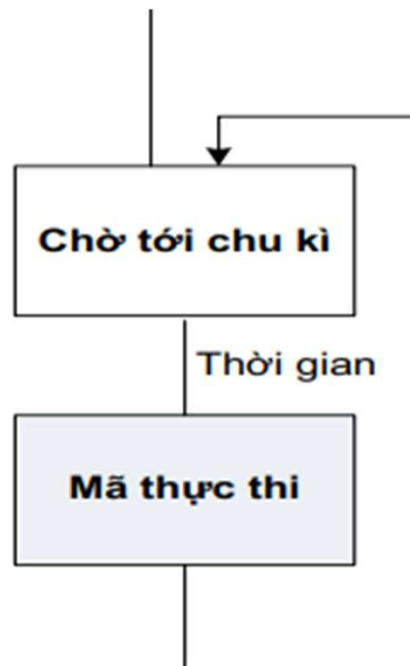
• Phân loại:



Tác vụ mặc định

Ví dụ:

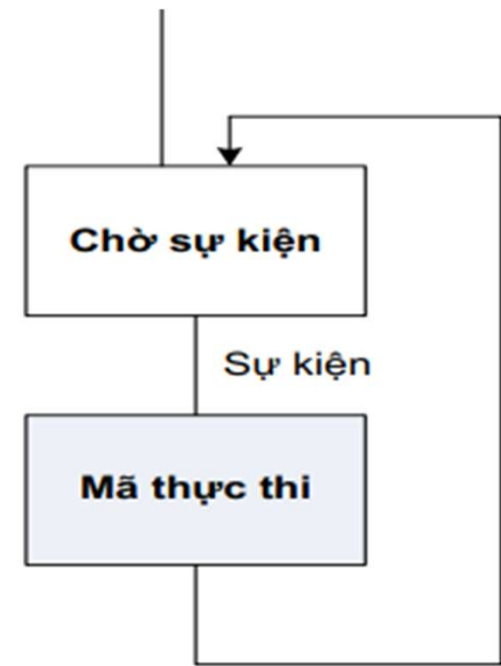
- Điều khiển logic
- Kiểm tra lỗi



Tác vụ tuần hoàn

Ví dụ:

- Điều chỉnh vòng kín (loop control)
- Xử lý truyền thông



Tác vụ sự kiện

Ví dụ:

- Điều khiển trình tự
- Xử lý sự cố

❖ Một số khái niệm:

■ Tác vụ (Task)

- hệ điều hành thực hiện một số cơ chế để đồng bộ thực hiện các tiến trình, bao gồm:
 - pipe (đường ống): không tên, FIFO: có tên;
 - thông điệp (message): cho phép các TT gửi các khuôn dữ liệu có khuôn dạng tới bất kì TT nào;
 - vùng nhớ chia sẻ (shared memory): các TT chia sẻ một phần không gian địa chỉ ảo của mình;
 - đánh tín hiệu (semaphore): các TT dùng để đồng bộ việc thực hiện.

❖ Một số khái niệm:

■ Cờ hiệu (Semaphore)

- Định nghĩa: Là một cơ chế đồng bộ hóa hoạt động như một "chiếc chìa khóa" để quản lý quyền truy cập vào tài nguyên dùng chung.
- Phân loại:
 - Semaphore nhị phân (Binary): Chỉ có hai trạng thái (0 và 1), dùng để loại trừ lẫn nhau (mutex) khi chia sẻ một tài nguyên duy nhất như máy in.
 - Semaphore đếm (Counting): Được khởi tạo với một giá trị nguyên dương, dùng để quản lý các hệ thống có nhiều tài nguyên cùng loại (ví dụ: bộ đệm nhớ).
- Thao tác: Sử dụng lệnh Pend (Acquire) để dành lấy chìa khóa và Post (Release) để trả lại chìa khóa cho các tác vụ khác.

❖ Một số khái niệm:

- Ngắt (Interrupt) và Chương trình dịch vụ ngắt (ISR)
 - Ngắt: Là một sự kiện làm dừng chương trình hiện tại để CPU chuyển sang thực hiện một tác vụ khẩn cấp khác.
 - Phân loại:
 - Ngắt cứng: Kích hoạt bởi các tín hiệu vật lý từ thiết bị ngoại vi (như ADC, UART), hoạt động theo cơ chế dự bộ và không xác định được thời điểm xảy ra.
 - Ngắt mềm: Là các lời gọi hàm đặc biệt (lệnh INT hoặc TRAP) được kích hoạt từ bên trong chương trình, hoạt động đồng bộ tại các thời điểm xác định.
 - ISR (Interrupt Service Routine): Đoạn mã xử lý riêng cho từng ngắt. Sau khi ISR kết thúc, CPU sẽ khôi phục trạng thái ban đầu và quay lại chương trình chính.

❖ Một số khái niệm:

■ Ngoại lệ (Exception)

- Định nghĩa: Là bất kỳ sự kiện nào làm gián đoạn việc thực thi bình thường của CPU và đẩy nó vào trạng thái đặc quyền để xử lý khẩn cấp.
- Phân loại:
 - Ngoại lệ đồng bộ: Phát sinh từ các sự kiện bên trong khi thực hiện lệnh CPU, ví dụ: chia cho 0, truy cập sai địa chỉ bộ nhớ.
 - Ngoại lệ không đồng bộ: Phát sinh từ bên ngoài, tương tự như ngắt cứng, ví dụ: ấn nút RESET trên hệ thống.

❖ Định nghĩa của hệ điều hành thời gian thực (Real Time Operating System - RTOS)

- Định nghĩa: là hệ điều hành đa tác vụ cho các ứng dụng với các ràng buộc (constraints) thời gian thực, được dùng trong lĩnh vực thiết kế các hệ thống nhúng.
- Ràng buộc về thời gian thực: yêu cầu một độ trễ nhất định cho một sự kiện và đáp ứng của hệ thống.

❖ Lợi ích của RTOS:

- Chia dự án thành các phần nhỏ hơn để ‘xử lý’
 - Mỗi vấn đề con sau khi phân chia sẽ trở thành một tác vụ. Mỗi tác vụ này thường chỉ thực hiện một công việc đơn giản và độc lập.
 - Về mặt logic, các tác vụ này chạy song song với nhau được điều phối bởi bộ lập lịch của RTOS.
- Dễ dàng phân chia công việc trong nhóm.
 - Các chương trình mô-đun này thường dễ mã hóa, gỡ lỗi và sửa đổi.

❖ Lợi ích của RTOS:

Một thuộc tính quan trọng của RTOS là khả năng tách biệt với ứng dụng. Nếu một chương trình bị “chết” hay hoạt động không hợp lệ, hệ điều hành thời gian thực có thể:

- Cô lập chương trình này
- Kích hoạt cơ chế phục hồi
- Bảo vệ các chương trình khác

❖ Cơ chế của RTOS:

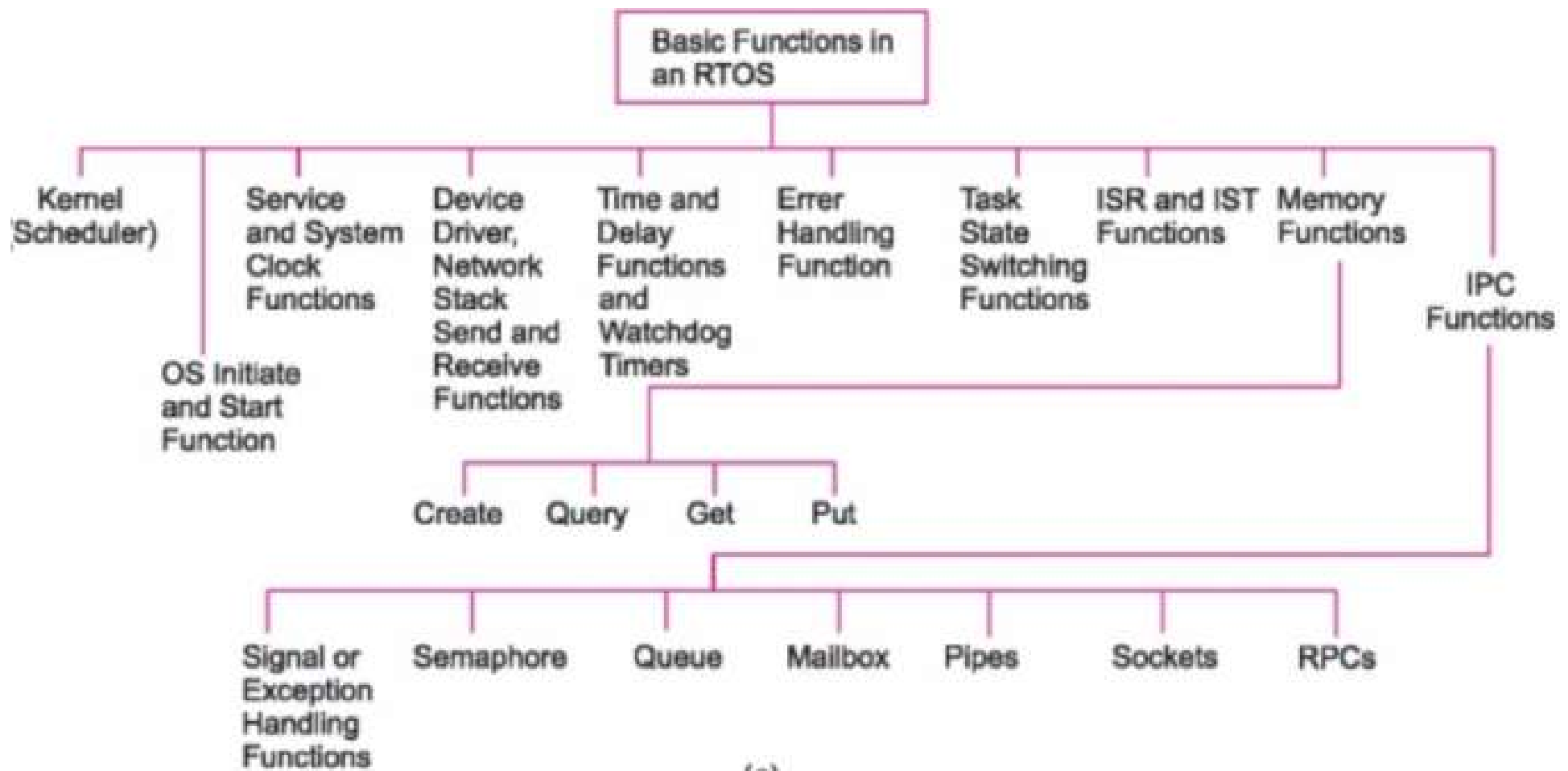
RTOS có nhiệm vụ giải quyết, điều phối các tác vụ, tạo bộ lịch và phân mức ưu tiên. Tất cả các việc này được thực hiện nhờ cơ chế:

- Hướng sự kiện (Event-driven): giải quyết, điều phối thông qua mức ưu tiên của tác vụ
- Chia sẻ thời gian (Time-sharing): chuyển đổi các tác vụ dựa trên cơ chế ngắt

❖ Cơ chế của RTOS:

Đặc điểm	Cơ chế Hướng sự kiện	Cơ chế Chia sẻ thời gian
Tác nhân kích hoạt	Sự kiện bên ngoài hoặc tín hiệu ngắt.	Xung đồng hồ hệ thống (Timer interrupt).
Nguyên tắc điều phối	Dựa vào mức độ ưu tiên của tác vụ.	Chia nhỏ thời gian CPU thành các nhát cắt.
Mục tiêu chính	Đáp ứng nhanh các tình huống khẩn cấp.	Đảm bảo tính đa nhiệm và công bằng tài nguyên.

❖ Các chức năng cơ bản của RTOS:

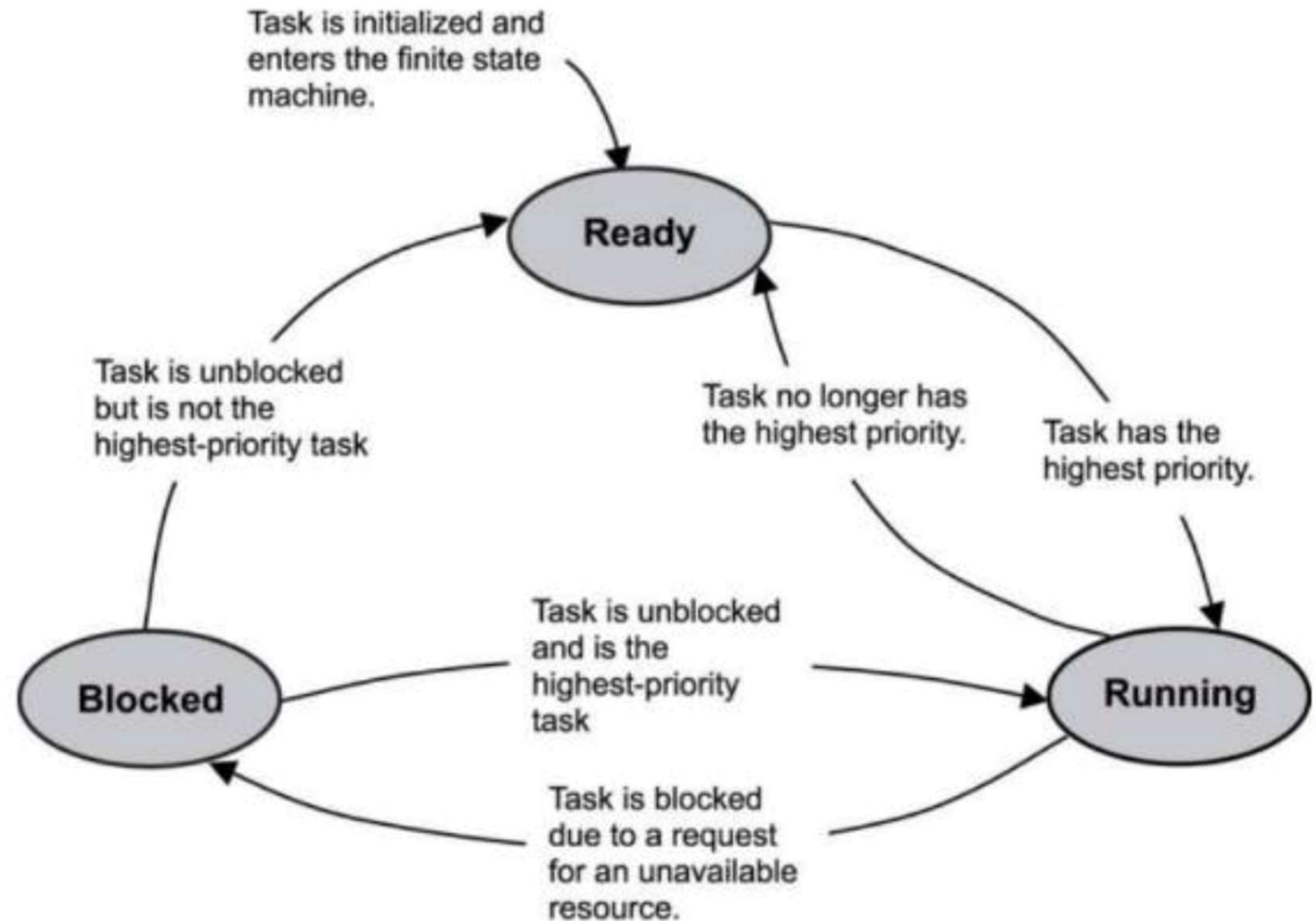


❖ Các chức năng cơ bản của RTOS:

- Bộ lập lịch
- Quản lý tiến trình
- Quản lý tài nguyên
- Quản lý thời gian
- Quản lý thiết bị
- Xử lý ngắt
- Quản lý bộ nhớ
- Quản lý hệ thống thiết bị ngoại vi I/O
- Quản lý thiết bị mạng

❖ Các chức năng cơ bản của RTOS:

- Bộ lập lịch



❖ Các chức năng cơ bản của RTOS:

- Bộ lập lịch là một module cốt lõi bên trong nhân hệ điều hành (kernel), đóng vai trò điều phối trình tự thực thi của các tác vụ (tasks) trên bộ vi xử lý.
 - Quản lý các tác vụ như những trạng thái máy (state machines: Ready, Running, Blocked/Wait) để quyết định khi nào một tác vụ được quyền sử dụng CPU.
 - Phân loại:
 - Lập lịch tĩnh: quyết định ngay từ giai đoạn thiết kế
 - Lập lịch động: quyết định trong quá trình thực thi
 - Lập lịch chiếm quyền: tạm dừng tác vụ có độ ưu tiên thấp hơn đang chạy để nhường CPU cho tác vụ có độ ưu tiên cao hơn
 - Lập lịch không chiếm quyền: các tác vụ được thực hiện liên tục

❖ Các chức năng cơ bản của RTOS:

- Các giải thuật lập lịch phổ biến
 - Rate Monotonic (RM): Gán mức ưu tiên dựa trên chu kỳ; tác vụ nào diễn ra càng thường xuyên thì càng được ưu tiên cao.
 - Earliest Deadline First (EDF): Tác vụ có hạn chót (deadline) gần nhất sẽ được ưu tiên thực hiện trước.
 - Round Robin (Vòng tròn): Mỗi tác vụ được cấp một khoảng thời gian ngắn (nhất cắt thời gian) để luân phiên sử dụng CPU, đảm bảo tính công bằng.
 - Shortest Job First (SJF): Ưu tiên các tác vụ có thời gian thực thi ngắn nhất để giảm thời gian chờ trung bình.

❖ Các chức năng cơ bản của RTOS:

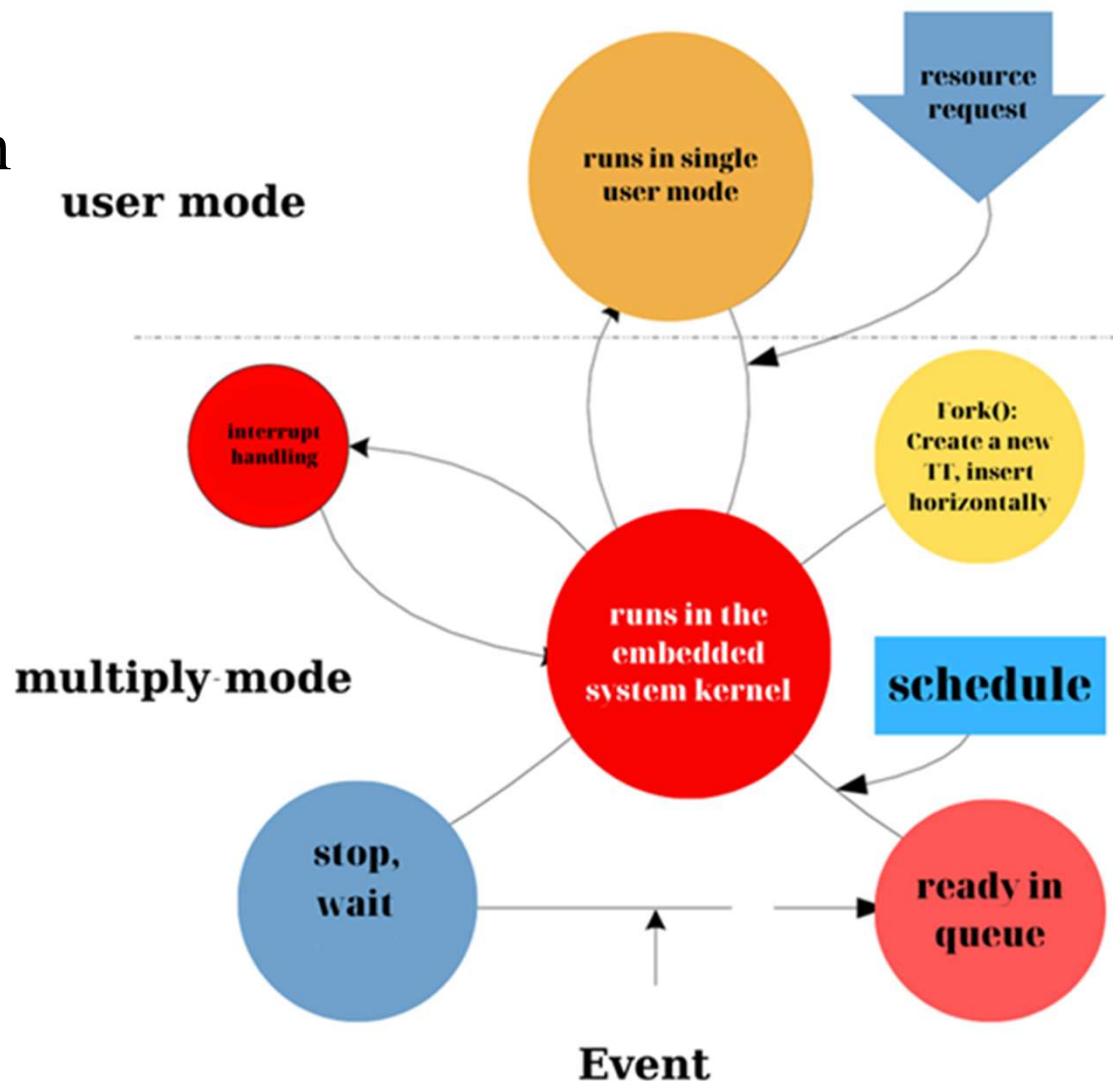
- Đồng bộ và thông điệp: các thông điệp được sử dụng để trao đổi thông tin giữa các hệ thống khác nhau hoặc giữa các task.

Các dịch vụ quản lý bao gồm:

- Semaphores: đồng bộ hoá quyền truy cập vào các tài nguyên dung chung
- Event flags: dùng để đồng bộ hoá các hoạt động cần có sự phối hợp của nhiều tasks
- Mailboxes, pipes, message queues: quản lý thông điệp đi-đến giữa các task

❖ Quản lý tiến trình:

- Trong hệ điều hành thời gian thực (RTOS), quản lý tiến trình thường được tập trung vào khái niệm tác vụ (task), là những đơn vị công việc nhỏ được chia ra từ một dự án lớn để xử lý đa nhiệm.
- Quá trình này giúp hệ thống phản ứng chính xác và kịp thời với các sự kiện ngẫu nhiên trong môi trường nhúng.



❖ Quản lý tiến trình:

■ Cơ chế Lập lịch và Điều phối

- Đây là "bộ não" của RTOS, đảm bảo tác vụ quan trọng nhất luôn được thực hiện trước.
- Lập lịch chiếm quyền (Preemptive): RTOS sẽ ngắt tác vụ ưu tiên thấp để nhường CPU cho tác vụ ưu tiên cao ngay khi nó sẵn sàng.
- Lập lịch vòng lặp (Round Robin): Các tác vụ có cùng mức ưu tiên sẽ luân phiên sử dụng CPU thông qua các nhát cắt thời gian (time slicing).
- Tính tiên định: Hệ thống phải đảm bảo thời gian phản ứng có thể dự đoán được và đáp ứng các hạn chót (deadline) khắt khe.

❖ Quản lý tiến trình:

- Đồng bộ và Truyền thông giữa các tác vụ (IPC)

RTOS cung cấp các dịch vụ để các tác vụ phối hợp và chia sẻ tài nguyên an toàn:

- Semaphore: Hoạt động như một "chiếc chìa khóa" để quản lý quyền truy cập vào tài nguyên dùng chung hoặc tạo tín hiệu cho sự kiện.
- Mailboxes & Message Queues: Cho phép các tác vụ gửi và nhận các bản tin hoặc khối dữ liệu có khuôn dạng để trao đổi thông tin.
- Monitor: Mô-đun đóng gói dùng để điều khiển việc truy cập vào vùng dữ liệu chia sẻ, đảm bảo loại trừ xung đột.

❖ Quản lý tiến trình:

- Các dịch vụ hỗ trợ của Nhân (Kernel)

Nhân RTOS chịu trách nhiệm thực hiện các tác vụ quản trị hệ thống cấp thấp:

- Chuyển đổi ngữ cảnh (Context Switching): Lưu trữ trạng thái của CPU khi tác vụ bị ngắt để có thể phục hồi chính xác sau đó.
- Quản lý ngắt: Nhận dạng và lựa chọn bộ xử lý phù hợp để phục vụ các dịch vụ ngắt (ISR) từ ngoại vi.
- Chó canh chừng (Watchdog Timer): Tự động phát hiện lỗi phần mềm hoặc tình trạng treo máy để khởi động lại hệ thống, đảm bảo tính sẵn sàng cao.

❖ Quản lý tiến trình:

- Thứ tự ưu tiên của các tiến trình:
 - Thứ tự ưu tiên của người dung: thứ tự ưu tiên tĩnh hoặc thứ tự ưu tiên thời gian thực;
 - Thứ tự ưu tiên thời gian thực cao hơn thứ tự ưu tiên tĩnh và thứ tự ưu tiên nhân rồi;
 - Các tác vụ ưu tiên nhân rồi chạy khi không có tác vụ ưu tiên cao nào đang chạy.

❖ Thiết kế hệ điều hành thời gian thực (RTOS)

- Một hệ thống nhúng với một CPU duy nhất chỉ có thể chạy một tiến trình tại một thời điểm.
- Một tiến trình tại bất kỳ thời điểm nào có thể được chạy bởi một hàm xử lý ngắt (ISR), một hàm nhân hệ điều hành hoặc một tác vụ.
- Có thể sử dụng luồng thay tiến trình:
 - Luồng (Threads): Là những phần chia nhỏ của một tiến trình, chạy trên các hệ có đa CPU, và được kiểm soát bởi hệ thống kiểm soát tiến trình.
 - Chạy các luồng trong nhân hệ điều hành để tăng tốc độ xử lý.
 - Chia sẻ không gian nhớ
 - Giảm thời gian chuyển đổi ngữ cảnh
 - Điều phối linh hoạt

❖ Thiết kế hệ điều hành thời gian thực (RTOS)

- Cung cấp khả năng xử lý hiệu quả các ISR, trình điều khiển, IST, luồng...;
- Bật và tắt ngắt;
- Cấp phát và giải phóng bộ nhớ tại các thời điểm và khối bộ nhớ cố định;
- Cung cấp khả năng lập lịch, chạy và dừng tác vụ hiệu quả trong trường hợp có nhiều tác vụ.

- ❖ Quản lý tiến trình thông qua can thiệp (hay còn gọi là chiếm quyền thực thi – preemption):
 - Hệ điều hành thời gian thực (RTOS) sẽ ngắt một tiến trình ưu tiên thấp khi một thông báo hoặc sự kiện đang chờ tiến trình ưu tiên cao;
 - RTOS có các điểm can thiệp ở cuối mã quan trọng, do đó RTOS có thể bị can thiệp tại các điểm này bởi các tác vụ ưu tiên cao trong thời gian thực;
 - Một phần nhỏ các chức năng của RTOS là không thể bị can thiệp.

❖ Quản lý tiến trình thông qua can thiệp (hay còn gọi là chiếm quyền thực thi – preemption):

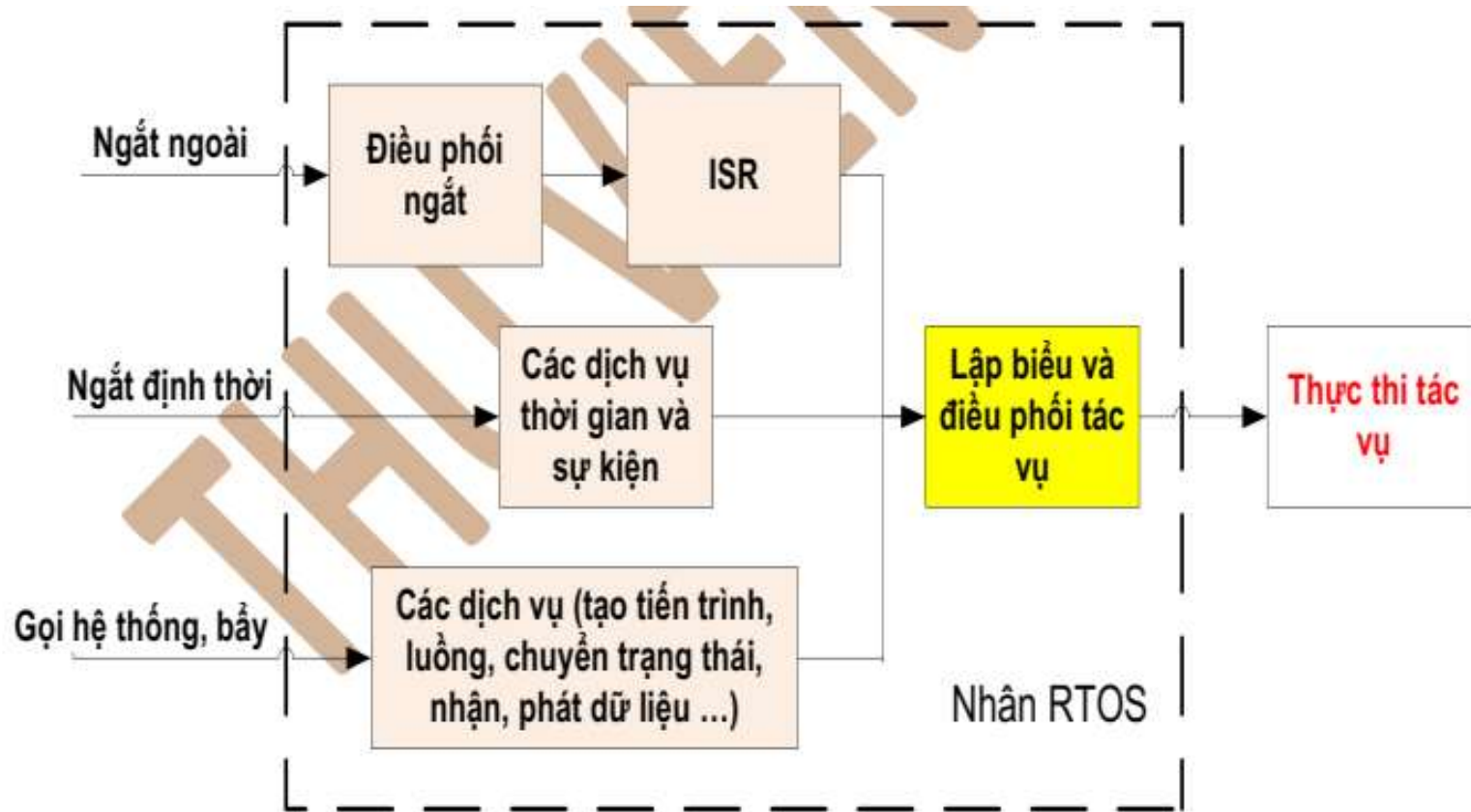
- Các chức năng không thể bị can thiệp
 - Vùng tới hạn (Critical Sections): Đây là các đoạn mã thực hiện những thao tác nhạy cảm như cập nhật cấu trúc dữ liệu của nhân (kernel), quản lý tài nguyên dùng chung hoặc thay đổi trạng thái các thanh ghi hệ thống,.
 - Cơ chế bảo vệ: Trong khi thực thi các chức năng này, RTOS có thể tạm thời vô hiệu hóa ngắt (CLI) để đảm bảo tính nguyên tử của thao tác, ngăn chặn việc dữ liệu bị xáo trộn hoặc gây lỗi hệ thống,.
 - Giới hạn thời gian: Để không ảnh hưởng đến tính thời gian thực, các đoạn mã không thể can thiệp này luôn được thiết kế cực ngắn và có giới hạn thời gian thực thi tối đa,.

❖ Quản lý tiến trình bằng cách kế thừa mức độ ưu tiên (kế thừa ưu tiên):

- Kế thừa ưu tiên cho phép chia sẻ tài nguyên với các tác vụ có mức độ ưu tiên thấp;
- Một tác vụ có mức độ ưu tiên trung bình sẽ không thể can thiệp vào một tác vụ có mức độ ưu tiên thấp trong khi nó đang được khóa để chạy cùng với tác vụ có mức độ ưu tiên cao;
- Giúp cải thiện hiệu suất hệ thống.
 - Đảm bảo tính tiên định (Predictability)
 - Đáp ứng thời hạn chót (Deadline)
 - Tối ưu hóa CPU

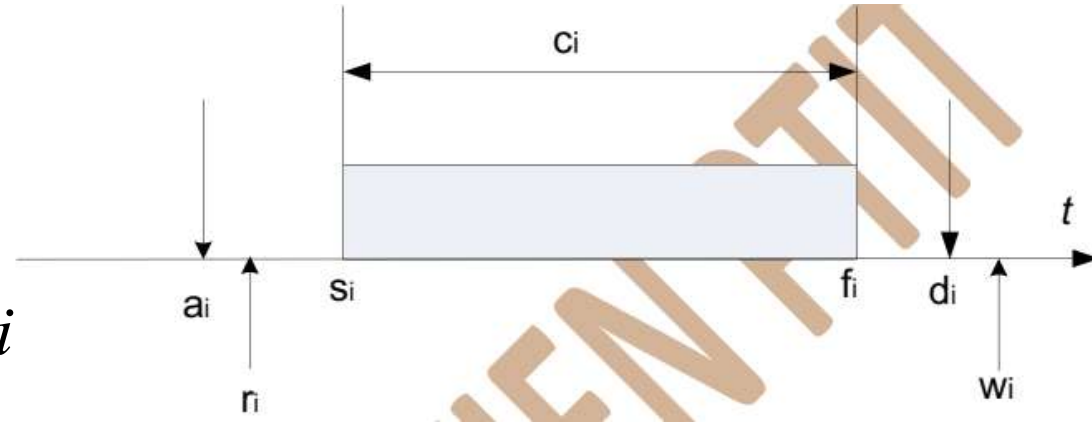
❖ Điều phối chạy chương trình bằng lập lịch (*schedule*):

- Lập lịch là một phép thực hiện phân bổ và gán quy trình thực thi các tác vụ cho bộ xử lý sao cho mỗi tác vụ được thực hiện hoàn toàn.
- Quản lý tác vụ thông qua lập biểu: Hầu hết các loại tác vụ đề xuyên qua lập biểu.



❖ Điều phối chạy chương trình bằng lập lịch (*schedule*):

- Thời điểm cuối cùng phải cho ra một đáp ứng trong các hệ ràng buộc về thời gian.
 - Thời gian xuất hiện a_i (*arrival time*)
 - Thời điểm bắt đầu thực thi r_i (*release time*)
 - Thời điểm bắt đầu thực hiện s_i (*starting time*)
 - Thời gian tính toán/thực thi c_i (*Computation time*)
 - Thời điểm hoàn thành f_i (*finishing time*)
 - Thời gian rủi ro/ xấu nhất w_i (*worst case time*) = $w_i - d_i$.
 - Thời điểm kết thúc d_i (*due time-deadline*): Thời điểm mà tác vụ phải hoàn thành.



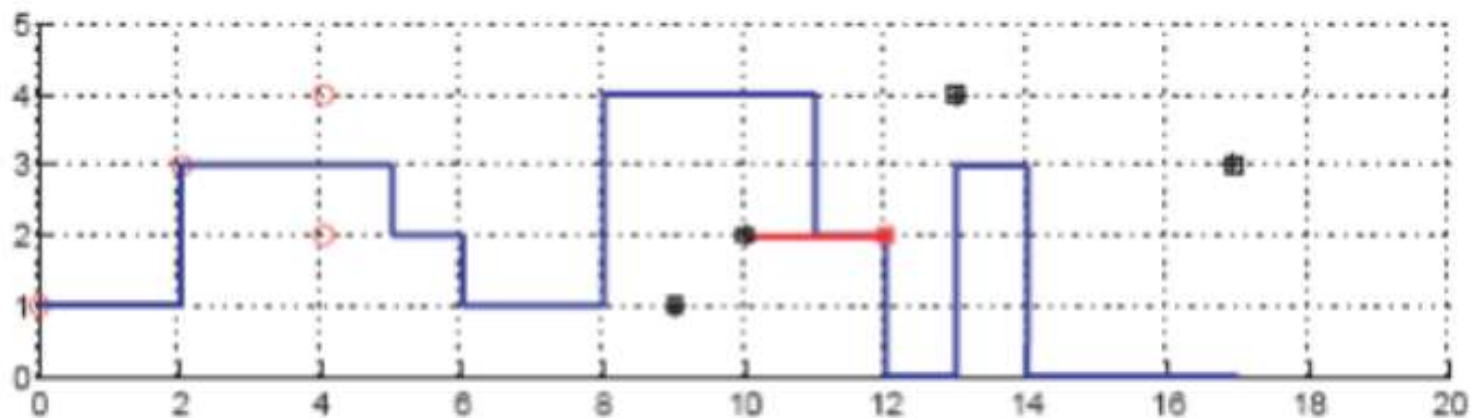
❖ Điều phối chạy chương trình bằng lập lịch (*schedule*):

- Ví dụ: Giả sử có 4 tác vụ với ai, di, ci như sau:

	J_1	J_2	J_3	J_4
<i>arrival time</i>	0	4	2	4
<i>deadline</i>	9	10	17	13

	J_1	J_2	J_3	J_4
C_i	4	2	4	3

Một kế hoạch vi phạm deadline:



❖ Điều phối chạy chương trình bằng lập lịch (*schedule*):

- Ví dụ: Giả sử có 4 tác vụ với ai, di, ci như sau:

	J_1	J_2	J_3	J_4
<i>arrival time</i>	0	4	2	4
<i>deadline</i>	9	10	17	13

	J_1	J_2	J_3	J_4
C_i	4	2	4	3

Thời gian trễ tối đa cho phép

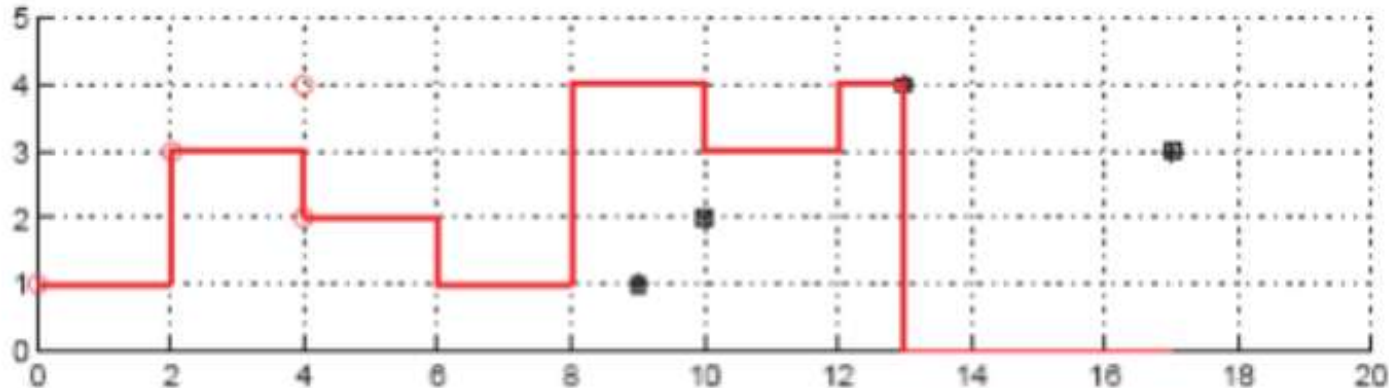
$$X1 = d1 - a1 - C1 = 9 - 0 - 4 = 5$$

$$X2 = 10 - 4 - 2 = 4$$

$$X3 = 17 - 2 - 4 = 11$$

$$X4 = 13 - 4 - 3 = 6$$

Một kế hoạch phù hợp:



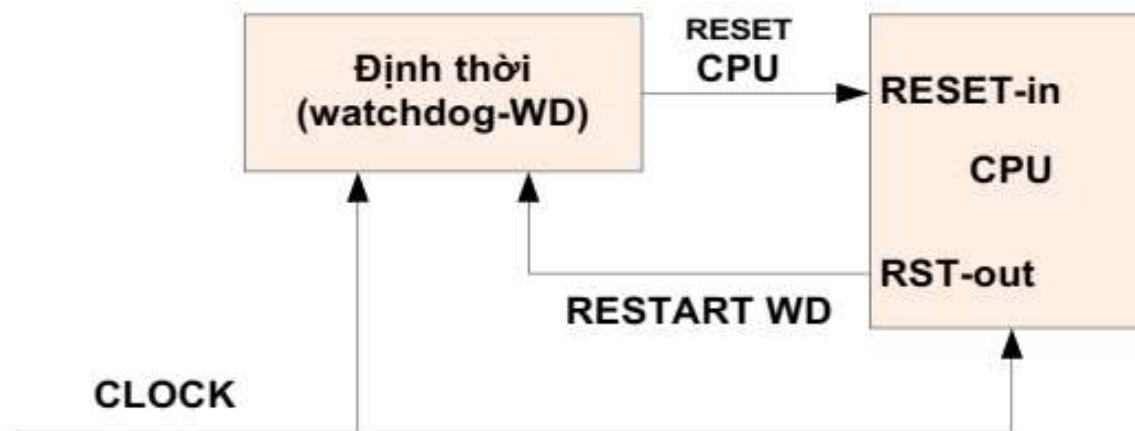
❖ Bộ giám sát định thời (Watchdog Timer - WDT)

- Định nghĩa: WDT là một bộ đếm (counter) có khả năng định nghĩa một khung thời gian hoạt động bình thường của hệ thống
- Phân loại:
 - WDT cứng: Là đồng hồ thời gian sử dụng các bộ đếm điện tử chuyên dụng tích hợp sẵn trong vi xử lý hoặc bo mạch.
 - WDT mềm: Là đồng hồ được lập trình bằng cách tăng/giảm giá trị đếm của một biến chương trình, phụ thuộc vào xung nhịp CPU.

❖ Bộ giám sát định thời (Watchdog Timer - WDT)

■ Nguyên lý hoạt động:

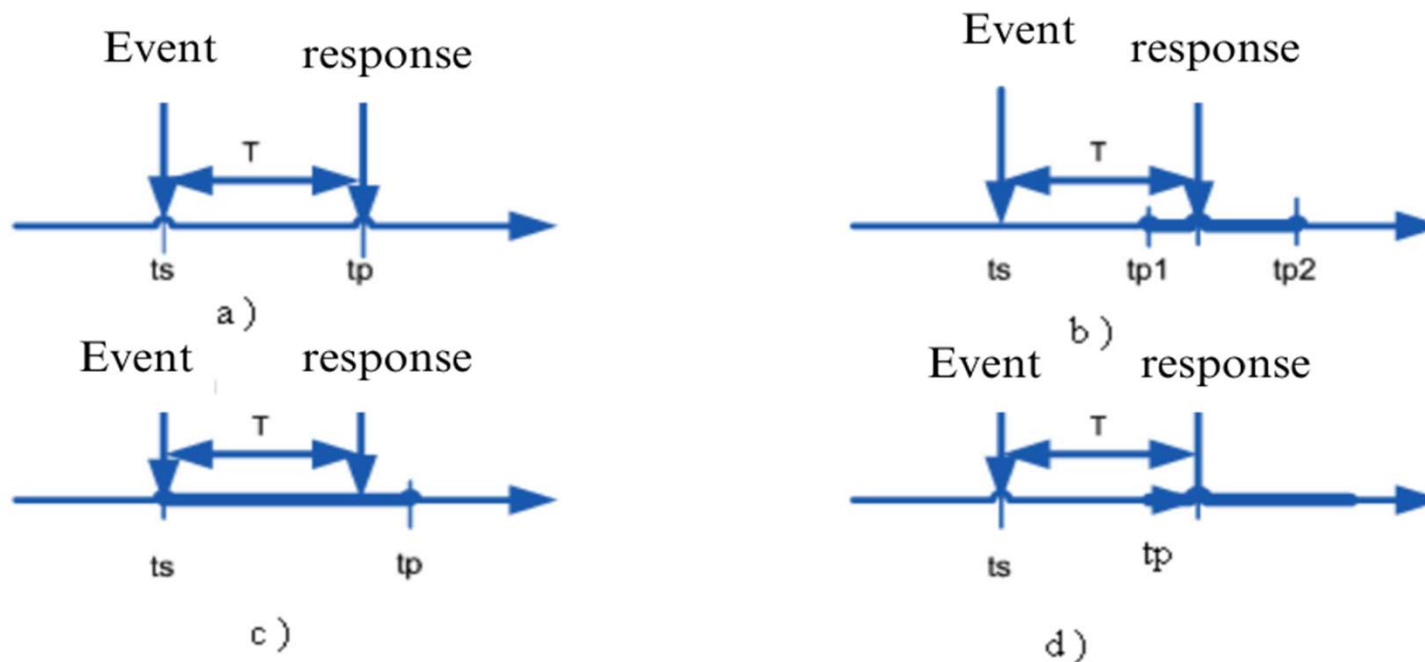
- WDT được nạp một giá trị định thời T . Nếu sau khoản thời gian T này mà bộ đếm không được tái khởi động, nó sẽ phát tín hiệu tràn (overflow).
- Trong điều kiện hoạt động bình thường, chương trình chính phải định kỳ thực hiện lệnh tái khởi động WDT
- Khi thời gian T bị vi phạm, WDT sẽ tự động kích hoạt một tín hiệu RESET cứng



- ❖ Hệ thống thời gian thực (Real-Time System - RTS) là hệ thống mà sự an toàn và tính đúng đắn của nó không chỉ phụ thuộc vào kết quả tính toán logic mà còn phụ thuộc vào thời điểm kết quả đó được đưa ra.
 - Một hệ thống có tính năng thời gian thực không nhất thiết phải có đáp ứng thực sự nhanh (càng nhanh càng tốt), mà quan trọng nhất là phải có phản ứng kịp thời và chính xác đối với các yêu cầu từ môi trường bên ngoài.
 - Đặc điểm cốt lõi
 - Tính bị động (Reactive): Hệ thống phải phản ứng với các sự kiện xuất hiện vào những thời điểm thường không biết trước
 - Tính chuẩn xác chức năng và thời gian: Các phép tính không chỉ cần đúng về mặt con số mà còn phải được thực hiện trong một chu kỳ thời gian đã xác định trước.
 - Tính đồng thời: Hệ thống có khả năng phản ứng và xử lý đồng thời nhiều sự kiện diễn ra cùng lúc.
 - Tính tiên định (Predictability): Người dùng có thể dự đoán trước được thời gian phản ứng chậm nhất và trình tự đưa ra các đáp ứng của hệ thống,.

❖ Hệ thống thời gian thực (Real-Time System - RTS)

■ Sự kiện và đáp ứng



❖ Hệ thống thời gian thực (Real-Time System - RTS)

- Sự kiện và đáp ứng
 - Các mốc thời gian:
 - ts (Time of Start): Thời điểm một sự kiện xảy ra từ thiết bị ngoại vi tác động tới hệ thống.
 - tp (Time of Peak/Response): Thời điểm hệ thống đưa ra đáp ứng đầu ra.
 - T (Interval): Khoảng thời gian hệ thống thực hiện các tính toán và xử lý ($T=tp-ts$).
 - Tùy vào đặc thù kỹ thuật của thiết bị, đáp ứng của RTOS có thể rơi vào các trường hợp sau:
 - Đáp ứng chính xác: Đáp ứng được đưa ra đúng thời điểm tp .
 - Đáp ứng trong khoảng: Đáp ứng xảy ra trong một cửa sổ thời gian xác định $[tp1, tp2]$.
 - Đáp ứng nhạy cảm thời gian: Đáp ứng phải xảy ra trong $[ts, tp]$.
 - Đáp ứng lỗi: Đáp ứng xảy ra sau thời điểm quy định

❖ Hệ thống thời gian thực (Real-Time System - RTS)

- Sự kiện và đáp ứng
 - Hệ thống nhúng xử lý sự kiện dựa trên hai mức độ ràng buộc:
 - Nhạy cảm với thời gian (Time-sensitive): Sự kiện phải được xử lý hoàn tất trong một khung thời gian nhất định.
 - Thời gian tới hạn (Time-critical): Khi có sự kiện, hệ thống phải phản ứng ngay lập tức bằng cách chuyển nhanh nhất đến mã chương trình xử lý sự kiện đó.
 - Để đảm bảo sự kiện luôn nhận được đáp ứng đúng hạn, RTOS sử dụng các công cụ quản lý mạnh mẽ:
 - Xử lý ngắt (Interrupt Handling): Sự kiện bên ngoài sẽ tạo ra một "yêu cầu ngắt" (IRQ). CPU tạm dừng chương trình chính để thực hiện "chương trình con dịch vụ ngắt" (ISR) nhằm đưa ra đáp ứng tức thời.
 - Bộ lập lịch (Scheduler): tại mọi thời điểm, chỉ có tác vụ có mức ưu tiên cao nhất (ứng với sự kiện quan trọng nhất) được thực thi để chiếm giữ CPU.
 - Chó canh chừng (Watchdog Timer): Giám sát các tác vụ

❖ Thiết kế hệ điều hành thời gian thực (RTOS)

- Quản lý I/O (thiết bị, tập tin, hộp thư...) trở nên dễ dàng hơn với hệ điều hành thời gian thực (RTOS).
- Quản lý hiệu quả nhiều trạng thái của CPU, các thiết bị như ổ cứng trong và ngoài, thiết bị ảo...
- Một hệ thống nhúng chạy hệ điều hành giúp đơn giản hóa việc làm việc với nhiều đầu vào (thông qua trình điều khiển) và xử lý dữ liệu dễ dàng hơn.

- ❖ Yêu cầu thiết kế hệ thống nhúng đa nhiệm phức tạp
 - Môi trường phát triển tích hợp (IDE)
 - Các chức năng đa nhiệm C/C++
 - Đồng hồ thời gian thực – bộ đếm thời gian
 - Bộ lập lịch (scheduler)
 - Trình điều khiển thiết bị, trình quản lý thiết bị
 - Các chức năng giao tiếp giữa các tiến trình, chức năng xử lý đa luồng...
 - Các chức năng khác: TCP/IP, cổng USB, mạng
 - Phần mềm kiểm thử và gỡ lỗi cho việc kiểm thử hệ điều hành thời gian thực (RTOS)

❖ Phân loại RTOS:

- Hard RTOS
- Soft RTOS
- Firm RTOS

❖ Phân loại RTOS:

- Hard RTOS: có khả năng đáp ứng các yêu cầu về thời gian thực trong mọi trường hợp.
 - Các ứng dụng sử dụng hard RTOS thường là thiết bị yêu cầu xử lý ngay lập tức
 - Sự sai sót trong việc tiếp nhận hạn chót dù chỉ một lần cũng có thể dẫn đến những hậu quả nghiêm trọng.
 - Nếu một tính năng xử lý ứng với một sự kiện nào đó không xảy ra hoặc xảy ra không đủ nhanh, Hard RTOS sẽ chấm dứt hoạt động đó để cô lập lỗi, bảo vệ độ tin cậy và tính sẵn sàng của phần còn lại của hệ thống.
 - Ví dụ: xe tự lái, hệ thống tự động trên máy bay, tên lửa ...

❖ Phân loại RTOS:

- Soft RTOS: đảm bảo đáp ứng các yêu cầu về thời gian thực nhưng có thể bị trễ trong một số tình huống nhất định.
 - Các kết quả tính toán hoặc đáp ứng của hệ thống nếu thu được sau thời hạn chót vẫn có ích và có giá trị nhất định (tính chất khấu hao giá trị)
 - Lỗi về mặt thời gian trong Soft RTOS chỉ dẫn đến việc giảm chất lượng dịch vụ hoặc giảm độ tin cậy của người dùng đối với hệ thống
 - Mặc dù chấp nhận trễ, các trễ trong nhân hệ điều hành (kernel delays) vẫn cần được nằm trong một ngưỡng giới hạn xác định để duy trì tính ổn định.
 - Ví dụ: ATM, UI, hiển thị, đa phương tiện,...

❖ Phân loại RTOS:

- Firm RTOS: là loại hệ điều hành nằm giữa Hard và Soft RTOS. Nó đảm bảo thực hiện các tác vụ theo thời gian quy định, tuy nhiên có thể có một số trường hợp bị lỗi hẹn

❖ Phân loại RTOS:

- Hệ điều hành thời gian thực (RTOS) do nội bộ phát triển
- Hệ điều hành thời gian thực thương mại

❖ Phân loại RTOS:

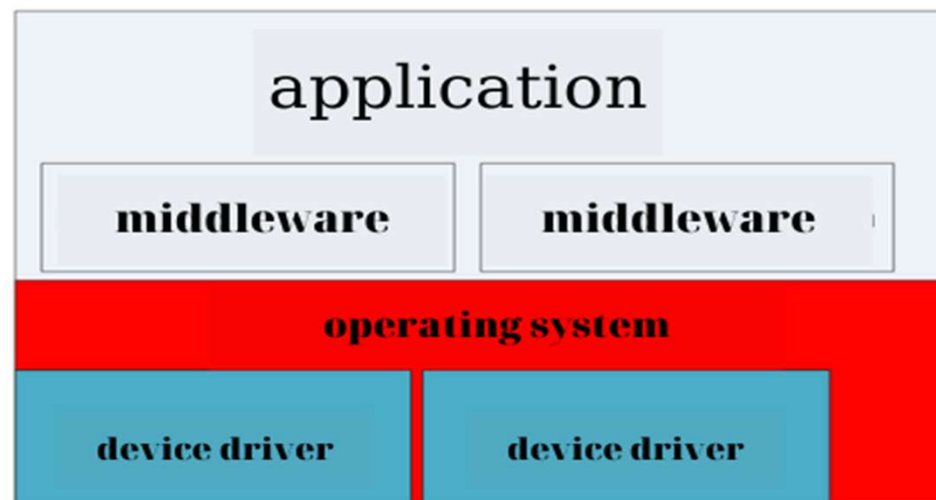
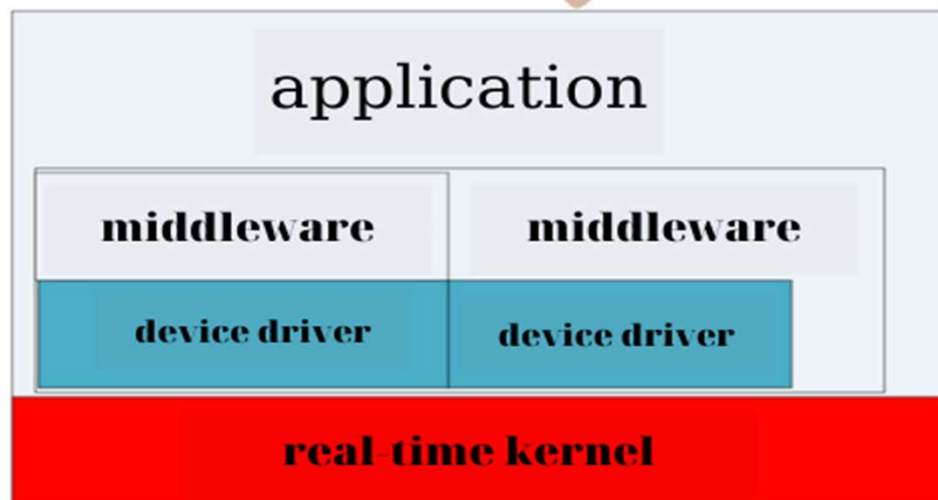
- Hệ điều hành thời gian thực (RTOS) do nội bộ phát triển
 - Mã được viết cho các yêu cầu ứng dụng hoặc sản phẩm cụ thể.
 - Tối ưu hóa chuyên biệt
 - Cứng hóa chức năng
 - Phát triển riêng biệt
 - Các yêu cầu có thể được tinh chỉnh.
 - Tái cấu hình linh hoạt
 - Tinh chỉnh qua các pha thiết kế
 - Tạo nhân mới

❖ Phân loại RTOS:

- Hệ điều hành thời gian thực thương mại
 1. Cung cấp các công cụ kiểm thử và gỡ lỗi.
 2. Hỗ trợ nhiều kiến trúc VXL (ARM, x86...)
 3. Hỗ trợ giao diện người dùng đồ họa (GUI)
 4. Hỗ trợ giao tiếp với nhiều thiết bị và nhiều giao thức kết nối khác nhau.
 5. Hỗ trợ phần mềm và thiết bị được tối ưu hóa.
 6. Đơn giản hóa quy trình lập trình của nhà phát triển.
 7. Tăng tốc quá trình phát triển sản phẩm.
 8. Tiết kiệm thời gian và chi phí bảo trì.

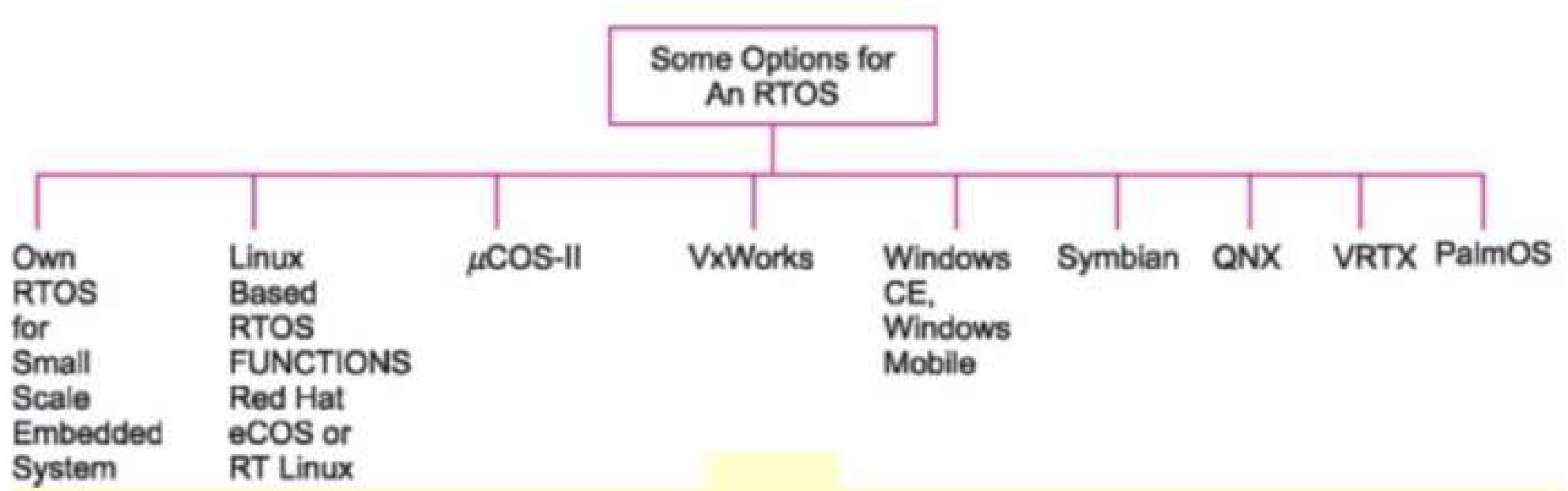
❖ Phân loại RTOS:

- RTOS nhân thời gian thực
- RTOS đa năng



❖ Phân loại RTOS:

- RTOS nhân thời gian thực
- RTOS đa năng



❖ Phân loại RTOS:

■ RTOS nhân thời gian thực

- các trình điều khiển thiết bị không phải là thành phần nhúng sâu bên trong nhân mà được đặt ngay phía trên nhân.
- Các thiết bị ngoại vi được quản lý bởi các **tác vụ độc lập**
- Hệ thống thường chỉ yêu cầu bộ định thời hệ thống (system timer) tích hợp trong nhân
- Nhân RTOS chịu trách nhiệm quản lý toàn bộ tài nguyên hệ thống và cung cấp các dịch vụ nền tảng cho phần mềm ứng dụng.

❖ Phân loại RTOS:

■ RTOS đa năng

- Linux, Windows OS.
- Có cấu hình giao diện đồ họa mạnh mẽ, nhiều giao diện đa phương tiện và giá thành thấp.
- Hỗ trợ giao diện đồ họa người dùng (GUI).

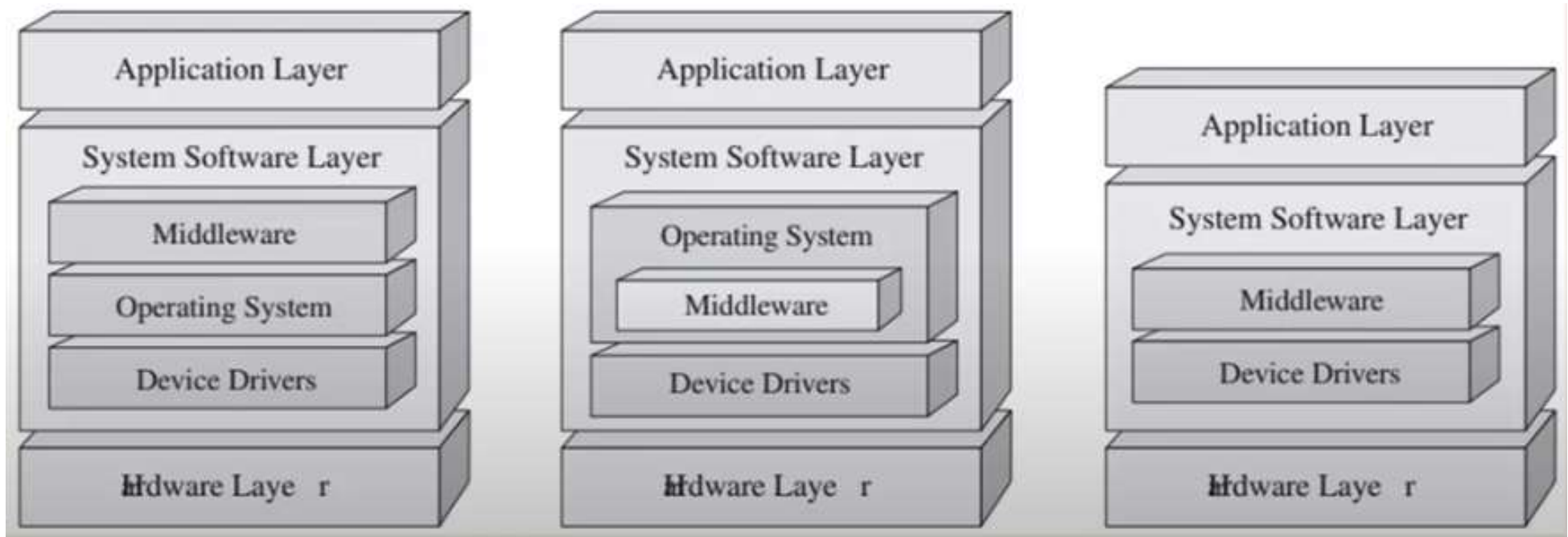
So sánh RTOS vs OS

- Hệ điều hành thời gian thực (RTOS) **ưu tiên các nhiệm vụ nhạy cảm về thời gian** và đảm bảo hoàn thành nhiệm vụ trong các giới hạn thời gian đã chỉ định. Trong khi hệ điều hành (OS) đa năng **tập trung vào hiệu suất tổng thể của hệ thống**.
- **RTOS** phục vụ các ứng dụng trong các **ngành quan trọng như hàng không vũ trụ, thiết bị y tế và hệ thống ô tô**, trong khi hệ điều hành đa năng (OS) phục vụ cho máy tính cá nhân và máy chủ.
- **RTOS** sử dụng thuật toán lập lịch xác định để đảm bảo các tác vụ quan trọng về thời gian thực thì có thể dự đoán được, không giống như hệ điều hành đa năng (**OS**) sử dụng thuật toán không xác định.

So sánh RTOS vs OS

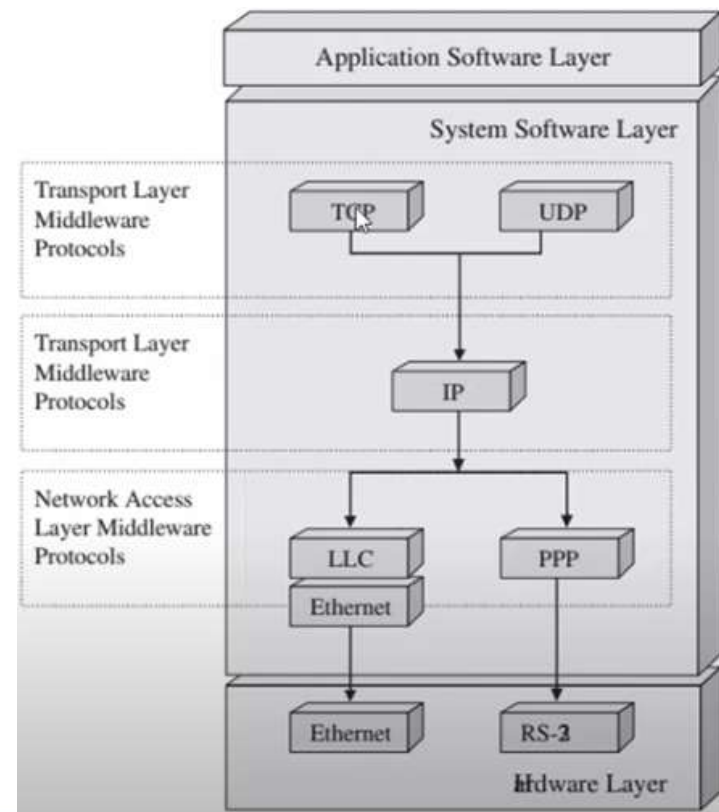
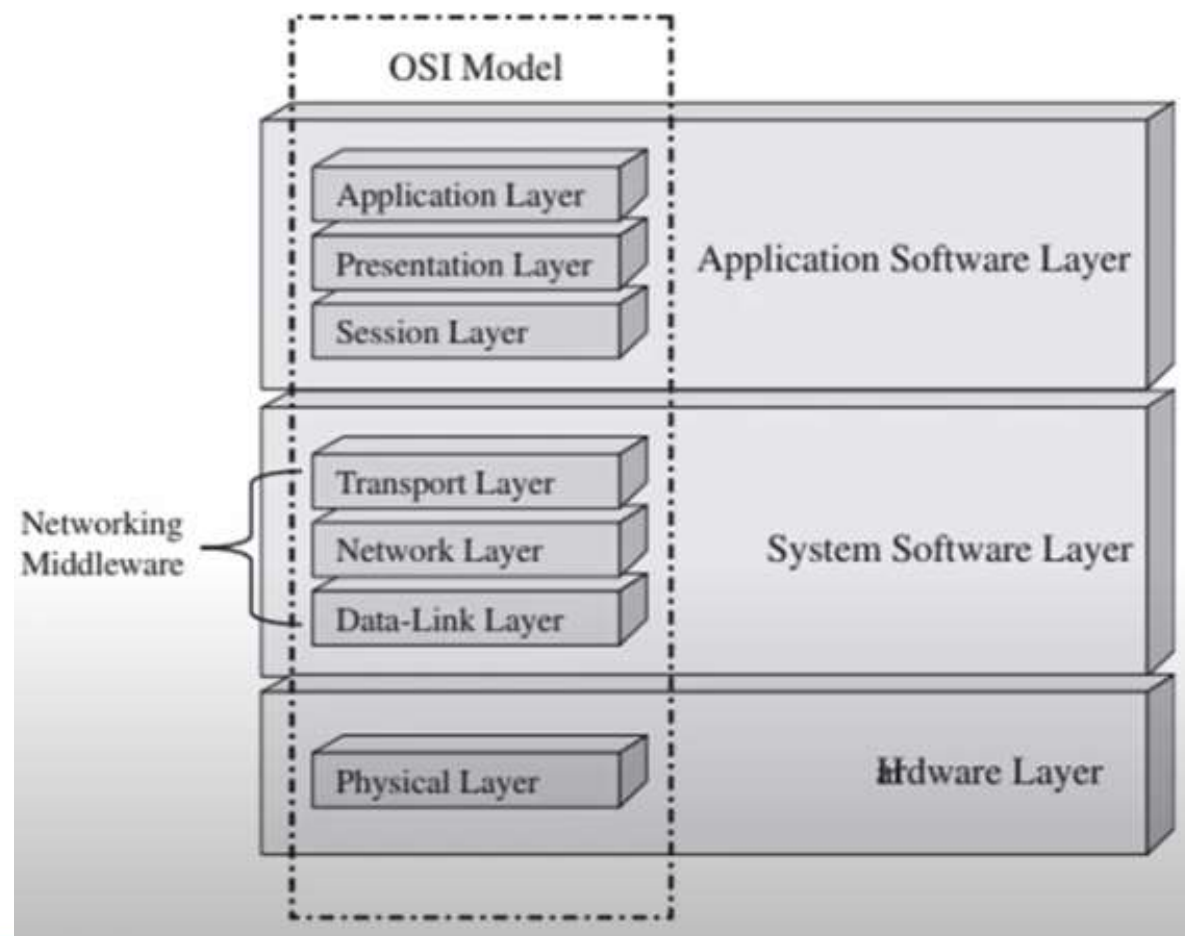
Characteristics	RTOS	OS
Complexity	Lightweight and designed for minimal overhead and reduced complexity.	More complex, supporting a wide variety of applications and hardware configurations.
Application Domain	Commonly used in domains where timing and determinism are critical, such as aerospace, automotive, industrial control systems, medical devices, and robotics.	Suitable for a broader range of applications, including desktop computers, servers, mobile devices, and consumer electronics.
Examples	FreeRTOS, VxWorks, QNX, eCos, and RT-Linux	Windows, macOS, Linux, and Unix variants
Resource Utilization	Optimized for minimal resource usage and efficient memory management.	Optimized for efficient resource utilization and user experience
Cost	High Cost	Low Cost
Determinism	Deterministic execution with guaranteed timing and deadlines	Non-deterministic execution with no guarantees of timing or deadlines
Real-Time Behavior	Provides deterministic behavior	Primarily focuses on multitasking and resource sharing.

- ❖ Middleware: nằm ở nhiều nơi, tách biệt với app, OS, driver.
- ❖ Application: phần mềm nằm ở lớp trên cùng, giao tiếp với người dùng.



❖ Middleware: xem như phần mềm trung gian, đảm bảo linh hoạt, an ninh và giúp liên kết các phần mềm ở các lớp kề cận.

- Chứa các hàm dịch vụ, được gọi sử dụng nhiều lần bởi các phần mềm khác.



- ❖ Phần mềm trung gian (Middleware - PMTG) đóng vai trò là một lớp phần mềm quan trọng nằm giữa hệ điều hành và các ứng dụng, cung cấp các dịch vụ dùng chung để đơn giản hóa quá trình phát triển và vận hành hệ thống nhúng.
 - PMTG là bất kỳ phần mềm nào không thuộc về nhân hệ điều hành (kernel), trình điều khiển thiết bị (TĐKTB) hay phần mềm ứng dụng.
 - Trong mô hình lớp xếp chồng của hệ thống nhúng, PMTG được coi như một phần của phần mềm hệ thống, đóng vai trò là cầu nối giữa các phần mềm hệ thống khác và cung cấp dịch vụ trực tiếp cho lớp ứng dụng.
 - PMTG thường không hợp nhất vào ứng dụng để có thể chia sẻ cho nhiều ứng dụng chạy đồng thời hoặc tái sử dụng trên các môi trường hệ điều hành khác nhau.

❖ Phần mềm trung gian (Middleware - PMTG)

- PMTG cung cấp các tiện ích nền tảng như an ninh hệ thống, kết nối mạng, truyền thông cục bộ giữa các ứng dụng và truy nhập cơ sở dữ liệu.
- Phân loại
 - PMTG đa năng (General-purpose): Được thiết kế để cài đặt trên nhiều loại hệ thống nhúng khác nhau, ví dụ như các giao thức mạng hoặc máy ảo Java (JVM).
 - PMTG xác định (Market-specific): Được thiết kế riêng cho các yêu cầu đặc thù của một thị trường hoặc thiết bị cụ thể, chẳng hạn như PMTG trên các máy thu hình kỹ thuật số hỗ trợ kết nối mạng.

❖ Phần mềm trung gian (Middleware - PMTG)

■ Các ví dụ điển hình

- Giao thức và khung kết nối: TCP/IP stack (lớp 2, 3, 4 của mô hình OSI), HTTP, CORBA, và Microsoft's .NET Compact Framework.
- Môi trường thực thi: Sun Embedded Java (JVM) là một PMTG phổ biến xuất hiện ở hầu hết các thiết bị nhúng hiện đại có hỗ trợ Web.
- Kiểu kiến trúc: PMTG hướng thông điệp (MOM), môi giới đối tượng (ORBs), và thủ tục gọi từ xa (RPCs).

❖ Phần mềm trung gian (Middleware - PMTG)

- Việc đưa thêm lớp phần mềm trung gian (PMTG) vào kiến trúc phần mềm của hệ thống nhúng mang lại nhiều tiện ích nhưng cũng tạo ra những thách thức cụ thể đối với hiệu năng thực thi:
 - Tạo thêm lớp xếp chồng kiến trúc: buộc các luồng dữ liệu và yêu cầu từ ứng dụng phải đi qua một bước trung gian trước khi đến được với hệ điều hành hoặc phần cứng,.
 - Mọi thay đổi hoặc xử lý trong lớp trung gian này đều có thể gây ra độ trễ lan truyền xuống các lớp bên dưới.
 - Đánh đổi giữa tính linh hoạt và tốc độ
 - Tăng chi phí xử lý hệ thống: việc quản lý ứng dụng và phần cứng trở nên phức tạp hơn, đòi hỏi tài nguyên CPU và bộ nhớ.

❖ Phần mềm ứng dụng (Application Software)

- Trong mô hình phân lớp, phần mềm ứng dụng nằm ở lớp cao nhất, phía trên lớp phần mềm hệ thống (HĐH, Monitor) và phần cứng.
- Đây là phần thực thi các bài toán chuyên biệt mà hệ thống nhúng hướng tới.
- Mỗi phần mềm ứng dụng thường được thiết kế để thực hiện một hoặc vài ứng dụng xác định (Application specific) cho từng miền thiết bị như robot, điện thoại, hay thiết bị y tế.

❖ Phần mềm ứng dụng (Application Software)

- Tùy vào loại hệ thống (kín hay mở), phần mềm ứng dụng sẽ bao gồm các mô-đun khác nhau:
 - Hệ thống nhúng kín (ví dụ: Bộ chuyển mạch mạng - Switch): Phần mềm ứng dụng bao gồm các mô-đun quản lý bộ đệm cổng, truy vấn địa chỉ MAC, chạy giải thuật chọn đường đi, quản trị thông lượng và Agent quản lý qua giao thức SNMP.
 - Hệ thống nhúng mở (ví dụ: Ngôi nhà thông minh): Bao gồm các ứng dụng liên quan đến Internet và giao diện người dùng như Web server, điều hướng (Navigation), e-mail, hoặc xem video theo yêu cầu (Video-On-Demand).
 - Hệ thống điều khiển: Thực hiện các thuật toán như bộ điều khiển PID dưới dạng mã chương trình để kiểm soát các tác nhân bên ngoài,.

❖ Phần mềm ứng dụng (Application Software)

- Phần mềm ứng dụng không trực tiếp điều khiển phần cứng mà thông qua các lớp trung gian:
 - Gọi hệ thống (System Call): Ứng dụng sử dụng giao diện lập trình API này để yêu cầu lớp phần mềm hệ thống thực hiện các dịch vụ như truy cập bộ nhớ, thiết bị ngoại vi hoặc mạng.
 - Luồng dữ liệu: Khi ứng dụng cần trao đổi dữ liệu, nó gọi chức năng của phần mềm hệ thống; phần mềm hệ thống chuẩn bị thông số gửi cho trình điều khiển thiết bị (Device Driver), sau đó nhận kết quả từ thiết bị trả về cho ứng dụng qua các vùng đệm.

❖ Phần mềm ứng dụng (Application Software)

- Đặc điểm trong Phát triển và Cài đặt
 - Tối ưu hóa: Mã nguồn ứng dụng phải có kích thước rất tối ưu vì dung lượng bộ nhớ (ROM) trên hệ thống nhúng thường hạn chế và yêu cầu thời gian xử lý tác vụ phải nhanh.
 - Phát triển chéo (Cross-development): Ứng dụng thường được viết và biên dịch trên máy chủ (Host) bằng ngôn ngữ bậc cao (như C/C++, Java), sau đó được nạp xuống hệ thống đích (Target) dưới dạng tệp ảnh thực thi (code image).
 - Tích hợp: Trong các hệ thống sử dụng RTOS, ứng dụng được chia thành nhiều tác vụ (Task) độc lập, chạy dưới sự giám sát của bộ lập lịch để đảm bảo các ràng buộc về thời gian thực.

- ❖ Lập trình nhúng là quá trình xây dựng các chương trình phần mềm, được biên dịch trên máy tính và sau đó nạp vào một hệ thống phần cứng chuyên biệt (thường là vi xử lý hoặc vi điều khiển) để điều khiển thiết bị hoạt động tự trị
 - Phần mềm được thiết kế để thực hiện một hoặc một vài ứng dụng xác định, chuyên biệt cho từng loại thiết bị.
 - Hệ thống nhúng thường có dung lượng bộ nhớ (RAM, ROM) giới hạn và năng lực tính toán của CPU không quá mạnh, đòi hỏi mã nguồn phải được tối ưu hóa.
 - Nhiều hệ thống yêu cầu phản ứng với sự kiện bên ngoài một cách nhanh nhạy, kịp thời và tuân thủ nghiêm ngặt các hạn chót (deadline)

❖ Ngôn ngữ lập trình phổ biến

- Ngôn ngữ C: Là ngôn ngữ phổ cập rộng rãi nhất và được coi là chuẩn trong thế giới nhúng nhờ khả năng can thiệp cấp thấp vào phần cứng nhưng vẫn dễ sử dụng.
- Embedded C (C nhúng): Một nhánh mở rộng của ngôn ngữ C, hỗ trợ đầy đủ các lệnh C gốc và bổ sung thêm các hàm/chỉ thị cụ thể để tương tác với bộ vi xử lý đích.
- Ngôn ngữ Assembly (Hợp ngữ): Sử dụng các từ gợi nhớ (mnemonics) để giao tiếp trực tiếp với CPU.
 - Ưu điểm: tối ưu hóa bộ nhớ và hiệu suất cực cao,
 - Nhược điểm: tốn thời gian phát triển và không thể chuyển đổi linh hoạt giữa các dòng chip khác nhau.
- VHDL/Verilog: Các ngôn ngữ mô tả phần cứng dùng để mô hình hóa hệ thống và thiết kế các vi mạch chuyên dụng (ASIC).

❖ Các kiến trúc phần mềm nhúng

Tùy vào độ phức tạp, phần mềm nhúng có thể tổ chức theo các cấu hình:

- Vòng lặp kiểm soát đơn giản (Super loop): Toàn bộ các tác vụ được đặt trong một vòng lặp vô hạn while(1), thực hiện tuần tự từ đầu đến cuối và lặp lại liên tục.
- Hệ thống ngắt điều khiển: Các tác vụ được kích hoạt dựa trên các sự kiện (ngắt) từ bộ định thời hoặc cổng giao tiếp ngoại vi, giúp hệ thống phản ứng tức thời hơn.
- Kiến trúc dựa trên RTOS: Sử dụng một Hệ điều hành thời gian thực để quản lý đa nhiệm, lập lịch cho các tác vụ (task) dựa trên mức độ ưu tiên, phù hợp cho các hệ thống phức tạp.
- Các thành phần quan trọng trong mã nguồn: Tác vụ (Task), Chương trình dịch vụ ngắt (ISR), Chó canh chừng (Watchdog Timer), Giao tiếp giữa các tác vụ (IPC)

❖ Phát triển phần mềm nhúng cần những gì?

1. Phần mềm biên dịch
2. Board phát triển
3. Bộ nạp/Bộ gỡ lỗi
4. Thư viện (cho lõi ARM, driver ngoại vi ...)
5. Tài liệu (datasheet)

❖ Các bước biên dịch phần mềm trong IDE (phần mềm biên dịch)

1. Tạo project
2. Cấu hình phần mềm biên dịch (chọn loại chip, chọn loại mạch nạp)
3. Viết chương trình và biên dịch
4. Nạp vào bộ nhớ flash của board phát triển
5. Chạy chương trình và kiểm tra lỗi

- ❖ Phát triển hệ thống dựa trên ngôn ngữ Hợp ngữ (Assembly) là một kỹ thuật lập trình cấp thấp, đóng vai trò quan trọng trong việc tối ưu hóa hiệu suất và kiểm soát trực tiếp phần cứng nhúng
 - Assembly là ngôn ngữ lập trình con người có thể đọc được bằng các từ gợi nhớ (mnemonics) thay vì dãy nhị phân 0 và 1 của ngôn ngữ máy,. Nó đóng vai trò là giao diện trung gian giữa bộ xử lý và lập trình viên
 - Mỗi loại bộ xử lý có một ngôn ngữ máy riêng, do đó mã Assembly chỉ có giá trị cho một kiến trúc cụ thể (ví dụ: Intel x86) và không thể tái sử dụng cho kiến trúc khác (ví dụ: như ARM)
 - Thường được dùng cho các tác vụ cấp thấp liên quan đến hệ thống, mã khởi động (boot code), trình điều khiển thiết bị (device drivers) hoặc khi cần tối ưu hóa tối đa tốc độ mà ngôn ngữ bậc cao không đáp ứng được

❖ Phát triển hệ thống dựa trên ngôn ngữ Hợp ngữ (Assembly)

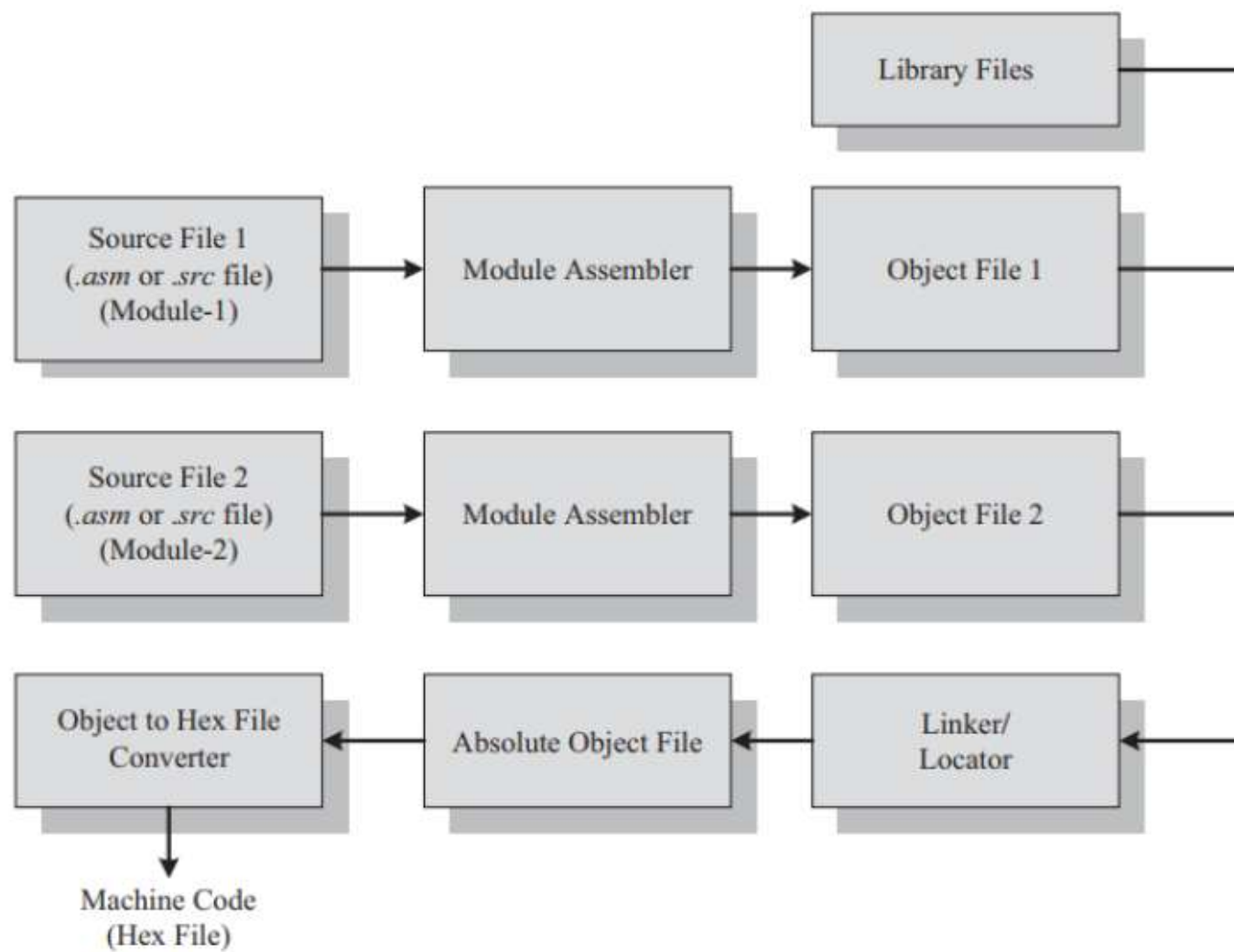
- Cấu trúc một lệnh Assembly: Một dòng lệnh Assembly điển hình thường được chia thành 4 trường: LABEL, OPCODE, OPERAND, COMMENTS,..
 - OPCODE (Mã lệnh): Cho bộ xử lý biết phải thực hiện hành động gì (ví dụ: MOV, ADD, INC),.
 - OPERAND (Toán hạng): Cung cấp dữ liệu hoặc thông tin vị trí để thực hiện lệnh. Có thể là toán hạng đơn, kép hoặc được ngầm định trong Opcode.
 - LABEL (Nhãn): Là bộ định danh giúp lập trình viên không phải nhớ địa chỉ tuyệt đối của dữ liệu hay mã nguồn.
 - COMMENTS (Chú thích): Bắt đầu bằng dấu chấm phẩy (;), giúp giải thích mã nguồn và bị trình biên dịch bỏ qua,.
 - Pseudo-ops (Chỉ thị giả): Các lệnh không tạo ra mã máy nhưng dùng để điều khiển trình biên dịch (ví dụ: ORG để xác định địa chỉ bắt đầu, EQU để gán giá trị cho biến).

❖ Phát triển hệ thống dựa trên ngôn ngữ Hợp ngữ (Assembly)

- Quy trình chuyển đổi sang Ngôn ngữ máy
 - Tập nguồn: Mã nguồn viết bằng Assembly được lưu dưới dạng .asm hoặc .src,.
 - Trình biên dịch (Assembler): Dịch các tập nguồn thành các tập đối tượng (.obj). Tại giai đoạn này, các địa chỉ vẫn là địa chỉ tương đối (có thể định vị lại).
 - Trình liên kết và định vị (Linker/Locator): Kết hợp các tập .obj và các thư viện (.lib), giải quyết các tham chiếu biến/hàm công khai (PUBLIC) và ngoại vi (EXTRN), sau đó gán địa chỉ bộ nhớ tuyệt đối.
 - Chuyển đổi sang tập Hex: Tạo ra tập .hex (định dạng Intel hoặc Motorola) chứa mã máy nhị phân cuối cùng để nạp vào bộ nhớ chương trình của vi điều khiển.

❖ Phát triển hệ thống dựa trên ngôn ngữ Hợp ngữ (Assembly)

- Quy trình chuyển đổi sang Ngôn ngữ máy



❖ Phát triển hệ thống dựa trên ngôn ngữ Hợp ngữ (Assembly)

- Ưu điểm:
 - Sử dụng bộ nhớ mã và dữ liệu cực kỳ hiệu quả
 - Mã nguồn được tối ưu hóa giúp tăng tổng hiệu suất thực thi của hệ thống.
 - Cho phép lập trình viên điều khiển mọi góc cạnh của kiến trúc bộ xử lý.
- Nhược điểm:
 - Khó lập trình, đòi hỏi người phát triển phải am hiểu sâu sắc về kiến trúc phần cứng chi tiết của CPU.
 - Do phụ thuộc vào tập lệnh của chip, nếu thay đổi bộ vi xử lý, toàn bộ ứng dụng phải được viết lại từ đầu,.
- Kết hợp Assembly và Ngôn ngữ bậc cao: Trong thực tế, người ta thường trộn mã Assembly vào mã C/C++ (Inline Assembly) để tận dụng ưu điểm của cả hai.
 - Assembly được dùng để viết các đoạn mã tới hạn về thời gian hoặc truy cập phần cứng cấp thấp,
 - C/C++ dùng để xử lý các logic phức tạp

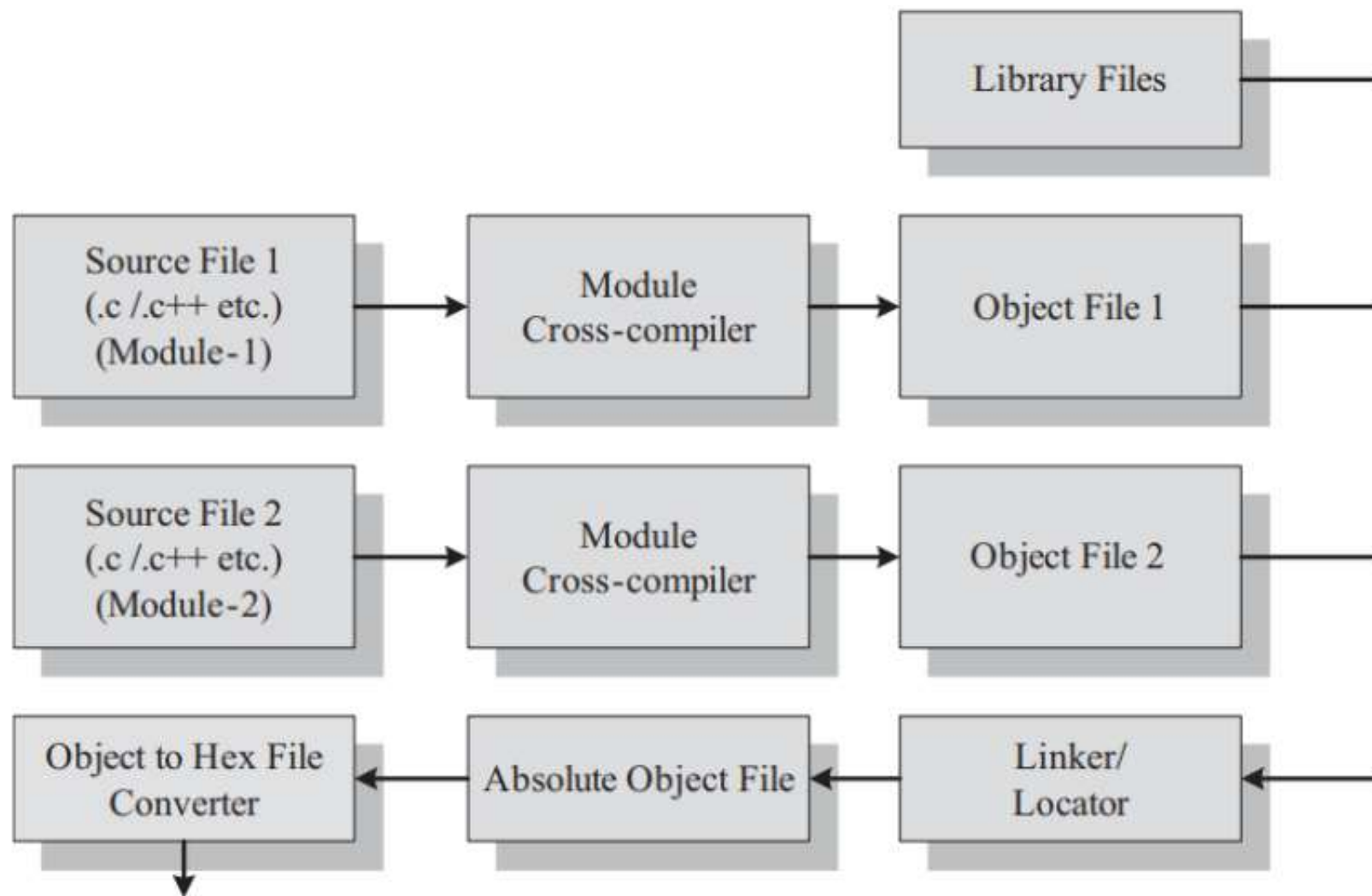
- ❖ Phát triển hệ thống nhúng dựa trên ngôn ngữ bậc cao (như C, C++, Java, Ada) là phương pháp phổ biến nhất hiện nay nhằm khắc phục các hạn chế về thời gian phát triển và tính phụ thuộc phần cứng của ngôn ngữ Hợp ngữ (Assembly).
 - Lập trình viên không cần am hiểu quá sâu các chi tiết phần cứng nội tại hay tập lệnh phức tạp của CPU. Các tác vụ được thực hiện với số dòng mã ít hơn so với Assembly.
 - Ứng dụng không bị bó buộc vào một kiến trúc vi xử lý cụ thể. Nếu thay đổi phần cứng (ví dụ từ Intel x86 sang ARM), lập trình viên chỉ cần sử dụng bộ biên dịch chéo tương ứng để dịch lại mã nguồn.
 - Các ngôn ngữ như C đã được chuẩn hóa (ANSI C) và có cộng đồng người sử dụng rộng lớn, giúp việc đào tạo và chuyển giao dự án dễ dàng hơn.

❖ Phát triển hệ thống nhúng dựa trên ngôn ngữ bậc cao

- Quy trình chuyển đổi sang Ngôn ngữ máy: Quá trình này diễn ra trên máy chủ (Host) thông qua một chuỗi các công cụ hỗ trợ để tạo ra tệp thực thi cuối cùng cho hệ đích (Target)
 - Viết mã nguồn: Sử dụng các trình soạn thảo văn bản để viết mã và lưu với phần mở rộng tương ứng (.c, .cpp, .java...).
 - Biên dịch chéo (Cross-compilation): Đây là bước then chốt. Một chương trình gọi là trình biên dịch chéo sẽ chuyển đổi mã nguồn bậc cao thành các tệp đối tượng (.obj) chứa mã máy của bộ xử lý đích.
 - Liên kết và Định vị (Linker/Locator): Trình liên kết sẽ kết hợp các tệp đối tượng và các thư viện (.lib), giải quyết các tham chiếu biến/hàm và gán địa chỉ vật lý tuyệt đối cho các khối mã và dữ liệu.
 - Tạo tệp nạp (Hex file): Chuyển đổi tệp đối tượng tuyệt đối thành định dạng nhị phân hoặc ASCII (như Intel Hex hoặc Motorola Hex) để nạp trực tiếp vào bộ nhớ của vi điều khiển.

❖ Phát triển hệ thống nhúng dựa trên ngôn ngữ bậc cao

- Quy trình chuyển đổi sang Ngôn ngữ máy:



❖ Phát triển hệ thống nhúng dựa trên ngôn ngữ bậc cao

■ Các ngôn ngữ tiêu biểu

- Ngôn ngữ C: Được coi là chuẩn mực trong thế giới nhúng vì là "ngôn ngữ bậc cao mức thấp nhất", cho phép điều khiển phần cứng trực tiếp mà không mất đi sức mạnh của ngôn ngữ bậc cao.
- Nhánh C nhúng (Embedded C) mở rộng thêm các chỉ thị cụ thể để tương tác với các tính năng đặc thù của vi xử lý.
- Ngôn ngữ C++: Kế thừa từ C, hỗ trợ tư duy hướng đối tượng và trừu tượng hóa dữ liệu. Tuy nhiên, nó có thể làm tăng chi phí thực thi và bộ nhớ nên thường dùng cho các dự án lớn.
- Java: Thường xuất hiện trong các thiết bị có hỗ trợ Web thông qua máy ảo Java (JVM).
- Ada: Ngôn ngữ hướng đối tượng có độ tin cậy rất cao, thường dùng trong quân sự và hàng không vũ trụ.

❖ Phát triển hệ thống nhúng dựa trên ngôn ngữ bậc cao

- Mặc dù mang lại sự tiện lợi, phát triển bằng ngôn ngữ bậc cao cũng có những điểm yếu cần cân nhắc:
 - Sử dụng ngôn ngữ bậc cao có thể làm mất đi khoảng 20% - 30% hiệu năng thực thi (tính theo MIPS - Microprocessor without Interlocked Pipeline Stages) so với việc tối ưu bằng Assembly.
 - Mã máy do trình biên dịch tự động tạo ra thường có kích thước lớn hơn và không tối ưu bằng mã do chuyên gia Assembly viết.
 - Sự cần thiết của trình biên dịch chéo: Mỗi ngôn ngữ bậc cao bắt buộc phải có trình biên dịch chéo hỗ trợ cho bộ xử lý đích thì mới có thể sử dụng được.

□ Trình biên dịch (Compiler) và Trình biên dịch chéo (Cross-compiler)

- Trình biên dịch (Compiler): Là một công cụ phần mềm có nhiệm vụ chuyển đổi chương trình được viết bằng ngôn ngữ thân thiện với con người (ngôn ngữ bậc cao như C, C++,...) thành tập mã máy tương đương mà bộ vi xử lý có thể hiểu và thực thi. Điểm đặc trưng của trình biên dịch thông thường là hệ điều hành, chương trình biên dịch và ứng dụng mã nguồn đều chạy trên cùng một bộ xử lý đích.
- Trình biên dịch chéo (Cross-compiler): Là loại trình biên dịch chạy trên một nền tảng phần cứng (máy chủ - Host) để tạo ra mã chương trình chạy trên một môi trường phần cứng khác (hệ đích - Target). Các trình biên dịch chéo thường linh hoạt, có thể cấu hình để tạo mã cho nhiều nền phần cứng khác nhau.

- ❖ C nhúng (Embedded C) là một nhánh mở rộng của ngôn ngữ lập trình C truyền thống, được thiết kế chuyên biệt để xây dựng các ứng dụng trên hệ thống nhúng
 - C nhúng hỗ trợ đầy đủ các lệnh của ngôn ngữ C tiêu chuẩn nhưng được bổ sung thêm các chức năng và chỉ thị cụ thể để tương tác trực tiếp với kiến trúc của bộ vi xử lý hoặc vi điều khiển đích
 - Đây là ngôn ngữ phổ cập nhất nhờ khả năng truy cập và điều khiển phần cứng trực tiếp hiệu quả mà vẫn giữ được tư duy của ngôn ngữ bậc cao
 - C nhúng có khả năng đáp ứng yêu cầu lập trình cho các bộ xử lý từ 8 bit đến 64 bit trong các hệ thống có tài nguyên bộ nhớ hạn chế

❖ C nhúng (Embedded C) có các đặc điểm quan trọng:

- Từ khóa volatile: Đây là một trong những từ khóa quan trọng nhất trong C nhúng. Nó thông báo cho trình biên dịch rằng giá trị của biến có thể thay đổi bất kỳ lúc nào do các tác nhân bên ngoài (như chương trình phục vụ ngắt - ISR, thanh ghi phần cứng hoặc luồng khác) mà không thông qua mã lệnh đang thực thi hiện tại.
- Thao tác bit: C nhúng hỗ trợ mạnh mẽ các phép toán bit (AND, OR, XOR, dịch bit) để kiểm soát các cờ trạng thái (flags) và điều khiển các chân vào/ra (I/O) của vi điều khiển.
- Quản lý bộ nhớ: Do tài nguyên nhúng rất hạn chế, lập trình viên cần tối ưu hóa kiểu dữ liệu.
- Vòng lặp vô hạn và Độ trễ (Delay): Hầu hết các ứng dụng nhúng chạy trong một vòng lặp vô hạn (while(1), for(;;) hoặc dùng goto) và thường xuyên sử dụng các hàm tạo độ trễ để chờ thiết bị sẵn sàng hoặc giao tiếp nối tiếp.

❖ C nhúng (Embedded C) có các đặc điểm quan trọng:

- Cấu trúc và Thành phần mở rộng
 - Chương trình phục vụ ngắt (ISR): C nhúng cung cấp các thuộc tính hoặc từ khóa đặc biệt (tùy thuộc vào trình biên dịch) để định nghĩa các hàm xử lý sự kiện ngắt từ phần cứng. Các hàm này thường không có tham số đầu vào và không trả lại giá trị.
 - Con trỏ (Pointers): Con trỏ được sử dụng rộng rãi để truy cập vào các địa chỉ bộ nhớ tuyệt đối được ánh xạ cho các chip ngoại vi hoặc thanh ghi phần cứng.
 - Cấu trúc (Structures) và Liên kết (Unions): Được dùng để quản lý dữ liệu cấu hình phức tạp hoặc trao đổi thông tin giữa ISR và ứng dụng chính.
 - Unions đặc biệt hữu ích trong việc tiết kiệm bộ nhớ vì nó cho phép các biến dùng chung một vùng nhớ tại các thời điểm khác nhau,.

- ❖ Keil C51 là một bộ công cụ phần mềm phổ biến được sử dụng rộng rãi trong phát triển hệ thống nhúng, cụ thể là cho họ vi điều khiển Intel 8051.
 - Đây là một trình biên dịch chéo (cross-compiler), cho phép người lập trình viết mã nguồn bằng ngôn ngữ C trên máy chủ (PC) và chuyển đổi thành mã máy có thể chạy được trên kiến trúc 8051 đích.
 - Trong môi trường phát triển tích hợp (IDE), Keil C51 giúp đơn giản hóa việc quản lý mã nguồn theo mô hình mô-đun.
 - bộ công cụ còn đi kèm với các phần mềm mô phỏng (simulator) và gỡ lỗi (debugger)
 - Keil C51 sử dụng cặp từ khóa `#pragma asm` và `#pragma endasm` để chèn các lệnh Assembly trực tiếp vào mã nguồn C

❖ Keil C51

- Các thành phần trong bộ công cụ Keil
 - C51 (Compiler): Trình biên dịch chéo cho ngôn ngữ C nhúng dành riêng cho họ 8051.
 - A51 (Macro Assembler): Bộ hợp ngữ dùng để dịch các tệp nguồn viết bằng ngôn ngữ Assembly sang mã máy.
 - LIB51 (Library Manager): Tiện ích dùng để tạo và quản lý các tệp thư viện (.lib), ẩn mã nguồn và tái sử dụng các hàm chức năng.
 - BL51 (Linker/Locator): kết hợp các mô-đun đối tượng (.obj) từ trình biên dịch và thư viện, giải quyết các tham chiếu biến/hàm và gán địa chỉ tuyệt đối cho bộ nhớ.
 - OH51 (Object to Hex File Converter): Công cụ chuyển đổi tệp đối tượng tuyệt đối thành tệp định dạng Intel Hex (.hex) để nạp vào bộ nhớ của vi điều khiển,.

❖ Keil C51 hỗ trợ nhiều tính năng chuyên biệt cho lập trình nhúng:

- Truyền tham số: C51 cho phép truyền tối đa ba đối số thông qua các thanh ghi mục đích chung từ R2 đến R7. Nếu số lượng đối số nhiều hơn ba, các đối số còn lại sẽ được chuyển qua các vị trí bộ nhớ cố định.
- Giá trị trả về: Các hàm trả về giá trị qua thanh ghi
 - ví dụ, giá trị kiểu char trả về qua R7 và kiểu int trả về qua cặp thanh ghi R7 và R6.
- Lập trình hỗn hợp (Inline Assembly): Keil C51 hỗ trợ chèn trực tiếp mã Assembly vào trong mã nguồn C bằng cách sử dụng cặp từ khóa `#pragma asm` và `#pragma endasm`.

- ❖ Keil C51 hỗ trợ nhiều tính năng chuyên biệt cho lập trình nhúng:
 - Xử lý ngắt (ISR): Trình biên dịch cung cấp từ khóa và cú pháp riêng để định nghĩa hàm dịch vụ ngắt,
 - ví dụ: `void interrupt_name (void) interrupt x using y.`
 - Thuộc tính này chỉ thị trình biên dịch tự động lưu trữ và khôi phục các thanh ghi quan trọng (ACC, B, PSW...) vào ngăn xếp (stack) khi xảy ra ngắt.